

INFORMATIQUE

2024 - 2025

Evaluation de Programmes

Nicolas VACELET

SUP^F_C

Evaluation de Programme Théorie de la Calculabilité et Algorithmique

Responsable de module: Nicolas VACELET

2024-2025

Table des matières

1	A propos	4
2	Introduction	5
2.1	Problème	5
2.2	Algorithme	6
2.3	Existe-t-il un algorithme pour chaque problème?	7
2.4	Tous les algorithmes sont-ils utilisables?	9
2.5	Complexité et algorithmique	10
3	Machine de Turing	11
3.1	Alphabet, mot, langage	11
3.2	Présentation des machines de Turing	12
3.2.1	Définition formelle d'une machine de Turing à un ruban . . .	12
3.2.2	Configurations d'une machine de Turing	13
3.2.3	Calculs d'une machine de Turing	14
3.2.4	Exemple	16
3.2.5	Mots et langages reconnus	17
3.3	Langages et fonctions décidables	18
4	Variations sur la machine de Turing	20
4.1	Machine à plusieurs têtes et plusieurs rubans	20
4.1.1	Surplace	20
4.1.2	Plusieurs têtes et rubans	21
4.2	Machine non-déterministe	25
4.3	Machine à 3 symboles	26
5	Décidabilité et indécidabilité	27
5.1	Thèse de Church	27
5.2	Langages et problèmes décidables	28
5.2.1	Propriétés des langages décidables	28
5.3	Problèmes indécidables	30

5.3.1	Codage d'une machine de Turing	30
5.3.2	Premier langage indécidable	31
5.4	Réductions et problèmes indécidables	32
5.4.1	Problème de l'acceptation ou problème universel	32
5.4.2	Problème de l'arrêt	32
6	Classes de complexité	34
6.1	Définitions de la complexité d'une machine de Turing	35
6.2	Définitions des classes de complexité	35
6.2.1	Comparaison de fonctions	36
6.2.2	Classe de complexité d'un problème	36
6.2.3	Propriété	37
6.2.4	Problèmes traitables	37
6.3	Thèse de l'invariance	37
7	Comparaisons d'algorithmes	38
7.1	Comparaison de fonctions	38
7.2	Complexité d'un algorithme	39
7.2.1	Opérations fondamentales	39
7.2.2	Nombre d'opérations	40
7.2.3	Complexité en moyenne et dans le pire des cas	44
8	Tri par comparaisons	46
8.1	Tri par sélection	47
8.1.1	Présentation	47
8.1.2	Analyse de la complexité en nombre de comparaisons	48
8.1.3	Analyse de la complexité en nombre d'affectations	49
8.2	Tri à bulles	49
8.2.1	Présentation	49
8.2.2	Analyse de la complexité en nombre de comparaisons	50
8.2.3	Analyse de la complexité en nombre d'affectations	51
8.3	Tri par insertion	52
8.3.1	Présentation	52
8.3.2	Analyse de la complexité en nombre de comparaisons	54
8.3.3	Analyse de la complexité en nombre d'affectations	54
8.4	Tri rapide	54
8.4.1	Présentation	54
8.4.2	Analyse de la complexité en nombre de comparaisons	59
8.5	État de l'art	59

9	Tri par dénombrement	60
9.1	Tri par dénombrement	60
9.1.1	Présentation	60
9.1.2	Analyse de la complexité	65
9.2	Application : tri par base	65
10	Problèmes $\mathcal{NP}$-complets	67
10.1	Introduction	68
10.2	Le Sudoku est un problème $\mathcal{NP}$	69
10.3	Présentation	69
10.3.1	Un algorithme $\mathcal{NP}$ pour le problème du Sudoku	70
10.3.2	Un algorithme déterministe	71
10.4	Transformations polynomiales	71
10.4.1	Définition	71
10.4.2	Propriétés	72
10.5	Problèmes $\mathcal{NP}$ -complets	72
10.6	SAT	73
10.6.1	Calcul booléen	73
10.6.2	Calcul propositionnel	74
10.6.3	Théorème de Cook	74
11	Rangs et médians	75
11.1	Avec un tri	75
11.2	Diviser pour régner	76
11.3	Faire des paquets	78
11.3.1	L'algorithme du médian	78
11.3.2	Complexité de l'algorithme du médian	79
12	Arbres binaires et tri par tas	82
12.1	Arbres binaires	82
12.1.1	Définitions	82
12.1.2	Taille et hauteur d'un arbre binaire	83
12.2	Les tas	84
12.2.1	Arbres partiellement ordonnés et tas	84
12.2.2	Deux opérations sur les tas	85
12.2.3	Propriété des tas	86
12.3	Tri par tas	87
12.3.1	Représentation d'un tas par un tableau	87
12.3.2	Algorithme du tri par tas	87
12.3.3	Analyse de la complexité en nombre de comparaisons	90

Chapitre 1

A propos

Ce document a été en grande partie rédigé par Claude Louis Canon et Anne Heam, Maître de Conférence à l'Université de Bourgogne Franche Comté.

Chapitre 2

Introduction

Contents

2.1	Problème	5
2.2	Algorithme	6
2.3	Existe-t-il un algorithme pour chaque problème ? . . .	7
2.4	Tous les algorithmes sont-ils utilisables ?	9
2.5	Complexité et algorithmique	10

Depuis la première moitié du XX^{ème} siècle, des logiciens comme K. Gödel, A. Church ou A. Turing ont cherché à définir la notion de **calcul**. En effet, certains théorèmes mathématiques énoncent l'existence d'un objet mais ne précisent pas comment l'obtenir. Par exemple : tout espace vectoriel admet une base. On peut alors se demander si l'objet peut effectivement se calculer. Cette question en amène une autre : qu'entend-on par calculable ? L'un des buts de ce cours est de définir ce qui est calculable. Pour cela, nous allons commencer par présenter les termes **problème** et **algorithme**.

2.1 Problème

Un **problème** en informatique est constitué de données sous une certaine forme et d'une question portant sur ces données.

Exemple 1

Problème 1

Données : 250 gr. de farine, 0,5 litre de lait, 2 œufs, 2 gr. de sel.

Question : Comment faire une pâte à crêpes ?

Problème 2

Données : Une voiture accidentée.

Question : Est-elle réparable ?

Problème 3

Données : $n \in \mathbb{N}$

Question : n est-il divisible par 10 ?

Problème 4

Données : Un échiquier avec une partie entamée et $n \in \mathbb{N}$.

Question : Est-ce que les blancs peuvent gagner en n coups ?

Une **instance** d'un problème est composée d'une valeur pour chaque donnée du problème.

Exemple 2

10 est une instance du problème 3, $(5 \times 3 + 4)$ en est une autre.

2.2 Algorithme

Une fois défini un problème, l'étape suivante consiste à essayer de le résoudre. Un **algorithme** est une méthode indiquant sans ambiguïté des actions mécaniques à effectuer pour trouver la réponse à un problème.

Précisons certains termes de cette définition : *sans ambiguïté* exprime le fait que tout le monde comprend la méthode de la même façon. Pour répondre au premier problème, on n'autorisera donc pas des instructions telles que "saler à votre convenance" car cela ne désigne pas la même quantité de sel suivant à qui on s'adresse. Le terme *mécanique* signifie qu'il ne fait pas appel à l'intelligence, à la réflexion.

Exemple 3

Voici un algorithme pour le problème 1 :

1. Mettre la farine dans une terrine.
2. Faire un puits.
3. Y casser les œufs entiers.
4. Ajouter l'huile, le sel et 10cl. de lait.
5. Travailler énergiquement la pâte avec une cuillère.
6. Mouiller progressivement avec le reste du lait.
7. Laisser reposer 1 heure.

Exemple 4

Voici un algorithme pour le problème 3 :

1. Déterminer le reste r de la division de n par 10.
2. Si $r = 0$, n est divisible par 10 sinon n n'est pas divisible par 10.

Un **programme** désigne alors la traduction d'un algorithme dans un langage de programmation.

Exemple 5

Voici un programme en C pour le problème 3 :

```
#include <stdio.h>
int main(){
    int n,r;
    printf("entrez votre nombre ");
    scanf("%d", &n);
    r=n%10;
    if (r==0){
        printf("%d est divisible par 10.\n", n);
    }
    else{
        printf("%d n'est pas divisible par 10.\n", n);
    }
}
```

2.3 Existe-t-il un algorithme pour chaque problème ?

Tout d'abord, quand on cherche un algorithme pour répondre à un problème donné, on s'aperçoit bien vite que la forme des données du problème a énormément d'importance. Par exemple, il est plus facile de savoir si un nombre est divisible par 10 s'il est écrit en base 10 que s'il est écrit en base 3 (par défaut, un nombre est donné en base 10). Cette remarque explique pourquoi la forme des données doit être explicitée dans la définition d'un problème.

Revenons à la question de ce paragraphe : existe-t-il un algorithme pour chaque problème ? La réponse à cette question est **non** : certains problèmes ne peuvent être résolus par aucun algorithme. Il est heureux pour la Française des Jeux qu'il n'existe aucun algorithme permettant de trouver le prochain tirage du loto. Voici un autre problème pour lequel il n'existe aucun algorithme :

Problème : problème de pavage

Données : un ensemble T fini de tuiles carrées dont les bords sont colorés et dont l'orientation est fixée.

Question : peut-on paver n'importe quelle surface avec des tuiles ayant uniquement des motifs appartenant à T de façon à ce que les couleurs de deux arêtes de tuiles qui se touchent soient les mêmes ?

Voici deux instances de ce problème : pour la première, on peut paver n'importe quelle surface avec des tuiles ayant uniquement des motifs appartenant à T de façon à ce que les couleurs de deux arêtes de tuiles qui se touchent soient les mêmes, pour la seconde non.

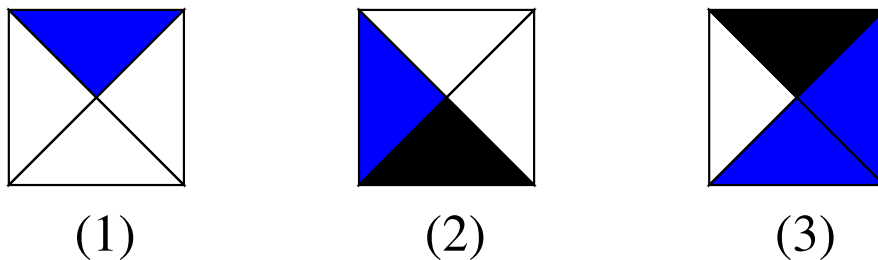


FIGURE 2.1 – Une instance du problème de pavage pour laquelle une solution existe.

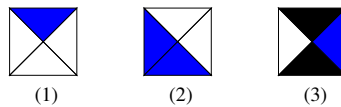


FIGURE 2.2 – Une instance du problème de pavage pour laquelle aucune solution n'existe.

Que signifie exactement le fait qu'aucun algorithme n'existe pour un problème donné ? Cela veut dire qu'il n'existe pas d'algorithme qui réponde à la question pour **n'importe quelle instance** du problème. C'est-à-dire que pour tout algorithme, il existe une donnée du problème telle que :

- soit l'algorithme ne s'arrête pas,
- soit il donne une réponse fausse.

Un problème pour lequel aucun algorithme n'existe est dit **indécidable**. Dans les années 30, A. Church avance une thèse appelée **Thèse de Church** qui peut s'énoncer de la façon suivante : il existe un algorithme répondant à un problème donné si et seulement s'il existe une machine de Turing qui détermine la solution de ce problème. Le prochain chapitre de ce cours présente les machines de Turing.

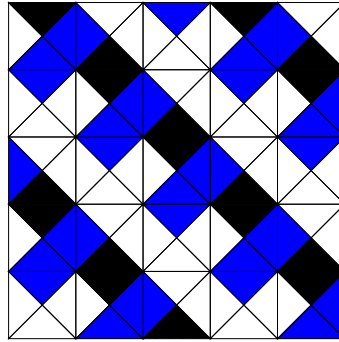


FIGURE 2.3 – Solution pour la première instance.

L'étude des problèmes décidables et indécidables constitue à elle seule une partie de l'informatique théorique appelée **calculabilité**. Ce module introduit ce domaine.

2.4 Tous les algorithmes sont-ils utilisables ?

La question qui vient naturellement quand on trouve un algorithme pour un problème est de savoir s'il est utilisable. Prenons l'exemple du puzzle des singes :

Problème : puzzle des singes

Données : un ensemble de cartes carrées dont l'orientation est donnée et dont les cotés représentent le bas ou le haut d'un singe

Question : peut-on arranger ces cartes afin de faire un grand carré tel que les moitiés se correspondent ?

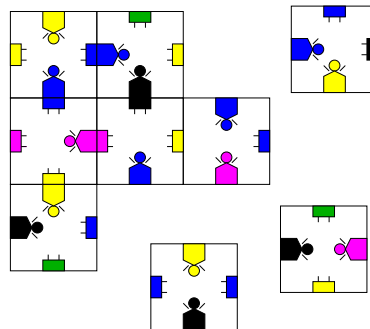


FIGURE 2.4 – Une instance du problème des singes.

Un algorithme naïf pour résoudre ce problème consiste à tester toutes les combinaisons de cartes jusqu'à en trouver une qui convienne ou jusqu'à avoir épuisé

toutes les possibilités. Étant donné que le nombre de combinaisons de cartes est fini, cet algorithme s'arrête.

Supposons que l'on ait 25 cartes, soit un carré de 5×5 , si aucune combinaison ne convient (le cas le pire), on peut être amené à toutes les tester. Cela fait $25!$ combinaisons différentes. $25!$ est un nombre qui comporte 26 chiffres. Un ordinateur qui vérifierait 1 milliard de combinaisons par seconde mettrait 490 millions d'années pour tester toutes les combinaisons ! Cet algorithme est donc inutilisable.

Il paraît donc pertinent de se poser une nouvelle question : existe-il un algorithme pour mon problème qui soit utilisable ? En cas de réponse positive, on dit que le problème est **traitable** (*tractable* en anglais).

Nous verrons un peu plus loin dans le cours où se situe la frontière entre traitable et non-traitable. Cependant, voici les deux principales raisons qui rendent un algorithme inutilisable :

- le temps de calcul est trop long
- il utilise trop de place mémoire

Ces deux problèmes sont liés. En effet, en un temps donné, on peut écrire un nombre fini d'informations en mémoire.

Il est important de déterminer qu'un problème est non-traitable car alors on peut rechercher deux types d'algorithmes utilisables :

- des algorithmes qui donnent une réponse approchée à la question
- des algorithmes qui ne répondent pas toujours

2.5 Complexité et algorithmique

Les théoriciens ont défini une mesure pour évaluer l'efficacité d'un algorithme, c'est-à-dire : son temps d'exécution et la place mémoire utilisée. Cette mesure est appelée la **complexité**. Elle permet de comparer deux algorithmes qui résolvent le même problème. L'étude de la complexité des problèmes et des algorithmes est appelée la **complexité**. Nous l'aborderons longuement dans ce module.

L'étude des méthodes pour améliorer la complexité d'un algorithme se nomme l'**algorithmique**. Nous ne l'aborderons pas dans ce cours mais nous en verrons un aperçu au travers des exercices.

Chapitre 3

Machine de Turing

Contents

3.1	Alphabet, mot, langage	11
3.2	Présentation des machines de Turing	12
3.2.1	Définition formelle d'une machine de Turing à un ruban	12
3.2.2	Configurations d'une machine de Turing	13
3.2.3	Calculs d'une machine de Turing	14
3.2.4	Exemple	16
3.2.5	Mots et langages reconnus	17
3.3	Langages et fonctions décidables	18

Les machines de Turing sont une abstraction des ordinateurs proposée par Alan Turing en 1936. Avant de les définir, il faut présenter un peu de vocabulaire.

3.1 Alphabet, mot, langage

En informatique théorique comme dans le langage courant, un **alphabet** désigne un ensemble fini de symboles : un **mot** d'un alphabet est une suite finie de symboles pris dans cet alphabet et un **langage** est un ensemble de mots. On ajoute à ces définitions celle du mot vide, noté ε (il désigne la suite vide) et la notation Σ^* pour l'ensemble des mots de l'alphabet Σ .

Enfin, on définit récursivement a^n par :

- $a^0 \stackrel{\text{def}}{=} \varepsilon$,
- $a^{n+1} \stackrel{\text{def}}{=} a^n a$.

Exemple 6

$\{a, bab\}$ est un langage

$\emptyset$ est un langage qui ne contient aucun mot. Il est distinct de $\{\varepsilon\}$ qui est le langage qui se réduit au mot vide.

$\{\omega \in \{a, b\}^* \mid \exists u, v \in \{a, b\}^* \text{ et } \omega = uabv\}$ est le langage des mots qui contiennent la sous-suite ab .

3.2 Présentation des machines de Turing

Une machine de Turing se résume à une tête de lecture comportant un nombre fini d'états internes et à un ruban. La puissance de l'analyse de Turing (1912-1954) tient au fait que sa tête de lecture ne lit qu'un seul symbole à la fois, et que cette lecture, associée à la table d'états adéquate, permet de connaître l'opération à effectuer.

La machine de Turing est une modélisation mathématique très simple, mais très puissante, du fonctionnement d'un ordinateur. La partie de contrôle représente le microprocesseur, elle est composée d'un nombre fini d'états. Le ruban représente la mémoire de l'ordinateur, il est infini. La tête de lecture représente la bus qui relie le microprocesseur à la mémoire. La figure 3.1 illustre ce modèle.

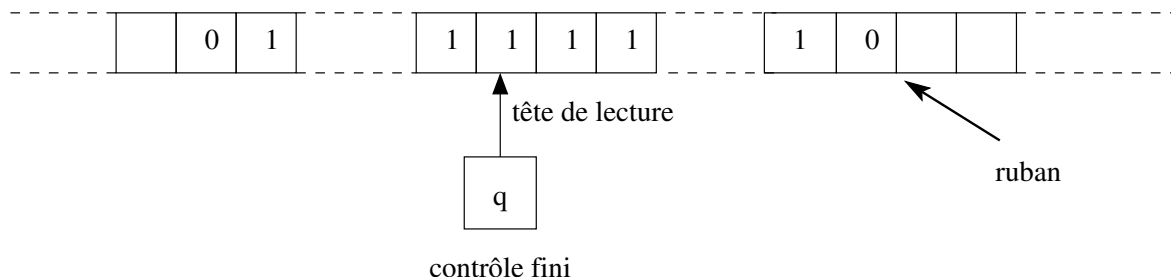


FIGURE 3.1 – Machine de Turing au cours d'un calcul.

Le calcul d'une machine de Turing est formé d'une suite d'étapes de calcul qui sont effectuées par la machine. À chaque étape de calcul, elle change d'état, remplace le symbole qu'elle voit sur le ruban et se déplace d'une case à gauche ou à droite. Une table de transitions indique quelles étapes de calcul sont possibles.

3.2.1 Définition formelle d'une machine de Turing à un ruban

Formellement, une **machine de Turing** M est un septuplet $(Q, q_0, F, \Gamma, B, \Sigma, \delta)$ tel que :

- Q est l'ensemble fini d'états du contrôle,

- q_0 est l'état initial,
- $F \subseteq Q$ est l'ensemble des états finaux,
- Γ est l'alphabet du ruban,
- $B \in \Gamma$ est un symbole spécial associé à la case vide (toutes les cases sauf un nombre fini contiennent B),
- $\Sigma \subseteq (\Gamma \setminus \{B\})$ est l'ensemble des symboles avec lesquels les mots fournis en entrée à la machine sont exprimés,
- δ est la fonction de transition de $Q \times \Gamma \rightarrow Q \times \Gamma \times \{G, D\}$.

Exemple 7

Voici une machine de Turing : $M = (Q, q_0, \{q_f\}, \{1, B\}, B, \{1\}, \delta)$, où $Q = \{q_0, q_i, q_p, q_f\}$ et δ est définie par le tableau suivant :

δ	1	B
q_0	$(q_i, 1, D)$	(q_f, B, G)
q_i	$(q_p, 1, D)$	
q_p	$(q_i, 1, D)$	(q_f, B, G)

La fonction de transition est représentée par un tableau. Il y a une ligne par état de contrôle (q_0, q_i, q_p, q_f) et une colonne par symbole de l'alphabet du ruban ($1, B$). La fonction n'est pas totale, pas définie partout (il n'y a pas de valeur dans la case correspondant à $q - i$ et B).

3.2.2 Configurations d'une machine de Turing

Le principe global de fonctionnement d'une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ est le suivant. Une entrée, c'est-à-dire un mot fini sur l'alphabet d'entrée Σ , est écrit sur la bande de la machine. Le reste de la bande est rempli avec le symbole blanc B . La tête de lecture est placée sur le premier symbole du mot d'entrée. Ensuite, la machine commence son calcul jusqu'à arriver éventuellement dans un état où elle accepte l'entrée (c'est-à-dire, un état appartenant à F). Pour décrire le fonctionnement précis d'une machine de Turing, il est nécessaire d'introduire la notion de configuration. Une configuration d'une machine de Turing est l'état global de la machine à un instant donné. Elle comprend

1. l'état de contrôle qui est un élément de Q ,
2. le contenu de la bande,
3. la position de la tête de lecture sur la bande.

Si la machine se trouve dans un état $q \in Q$, la **configuration** est écrite $\alpha q \beta$ avec $\alpha, \beta \in \Gamma^*$ où α est le contenu de la bande (strictement) à gauche de la tête de lecture et β est le contenu de la tête à droite de la tête de lecture comme illustré par la figure 3.2.

Le symbole sous la tête de lecture est donc le premier symbole de β . Quand on décrit une configuration de la machine, les symboles blancs qui se trouvent dans la partie infinie à droite et à gauche sont la plupart du temps omis. Ceci permet de décrire une configuration de façon finie. Dans la figure 3.2, la machine se trouve dans l'état q , sa configuration est $a_1 \dots a_{k-1} q a_k \dots a_n$ parce que le contenu de la bande avant la tête de lecture est $a_1 \dots a_{k-1}$ précédé de symboles blancs et que le contenu après la tête de lecture est $a_k \dots a_n$ suivi de symboles blancs.

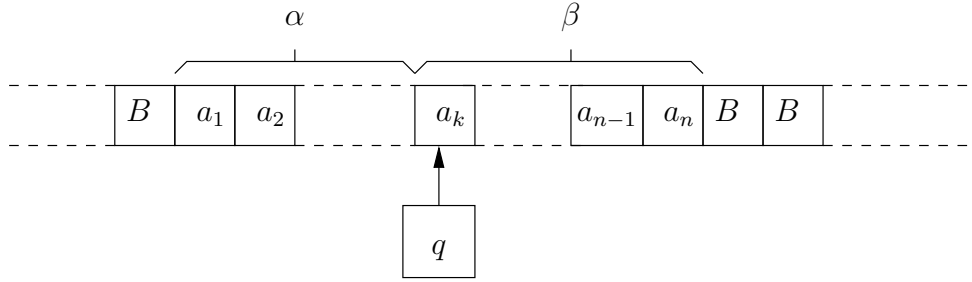


FIGURE 3.2 – Configuration d'une machine de Turing.

Au départ, la bande contient la donnée initiale et la tête de lecture se trouve sur la première position de la bande. La **configuration initiale** s'écrit donc $q_0\beta$ où q_0 est l'état initial et $\beta \in \Sigma^*$ est la donnée initiale.

3.2.3 Calculs d'une machine de Turing

Un calcul d'une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ se décompose en étapes. Une étape d'un calcul consiste à passer d'une configuration à une autre en appliquant une des transitions de la fonction de transition δ . Une étape de calcul comprend les trois actions suivantes.

1. changer l'état de contrôle
2. écrire un symbole à la place du symbole sous la tête de lecture
3. déplacer sa tête de lecture d'une position vers la gauche ou la droite

Une transition de la forme $(q, a) \xrightarrow{\delta} (q', b, x)$ peut uniquement être appliquée si la machine se trouve dans l'état q et si le symbole sous la tête de lecture est a . Après application de cette transition, la machine se trouve dans l'état q' , le symbole a est remplacé sur la bande par b et la tête de lecture se déplace d'une position vers la gauche si $x = G$ (Gauche) ou d'une position vers la droite si $x = D$ (Droite). On note $c \vdash_M c'$ une étape de calcul qui fait passer la machine d'une configuration c à une configuration c' en appliquant une transition de δ .

Formellement, étant donnée une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$, la **relation de transition** $\vdash_M$ sur les configurations de M est définie par :

- $\alpha qa\beta' \vdash_M \alpha bq'\beta'$ si $(q, a) \xrightarrow{\delta} (q', b, D)$
- $\alpha'a_{k-1}qa\beta \vdash_M \alpha'q'a_{k-1}b\beta$ si $(q, a) \xrightarrow{\delta} (q', b, G)$

Ces deux types de transition sur les configurations de M sont illustrés par les figures 3.3, 3.4 et 3.5.

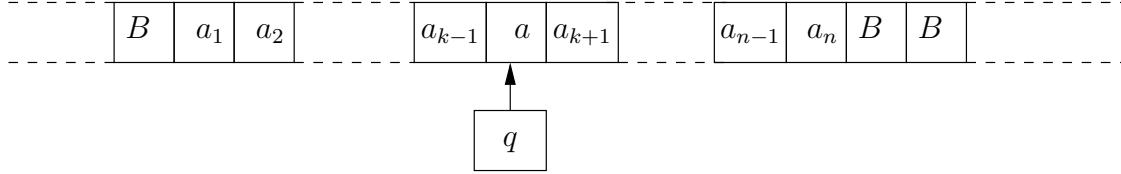


FIGURE 3.3 – Configuration de la machine de Turing avant la transition.

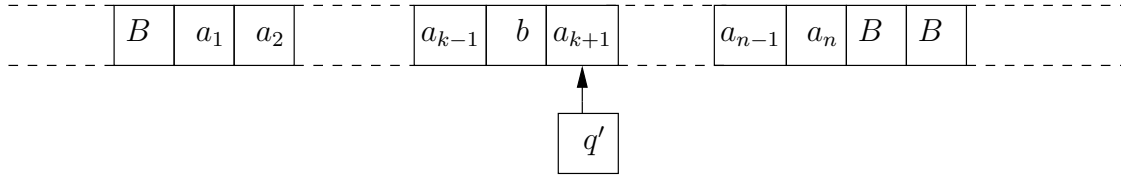


FIGURE 3.4 – Configuration après la transition si $(q, a) \xrightarrow{\delta} (q', b, D)$.

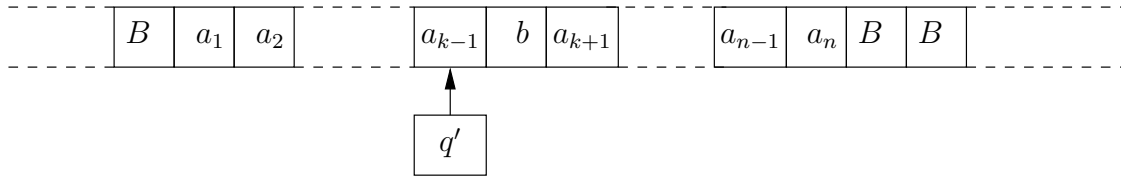


FIGURE 3.5 – Configuration après la transition si $(q, a) \xrightarrow{\delta} (q', b, G)$.

Étant donnée une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$, un **calcul** de M sur la donnée α est une suite de configurations $q_0\alpha \vdash_M c_1 \vdash_M \dots c_n \vdash_M \dots$ où les c_i sont des configurations.

Le calcul d'une machine de Turing sur le mot $\alpha \in \Sigma^*$ se déroule à partir de la configuration initiale, où la machine se trouve dans l'état q_0 , le ruban contient α sur des cases contiguës, les autres contiennent B et la tête de lecture pointe sur la case du ruban contenant le premier symbole de α . Tant que la fonction δ est définie sur l'état courant et le symbole sur lequel la tête de lecture pointe, la machine utilise δ pour passer d'une configuration à la suivante. Le calcul peut donc ne jamais s'arrêter.

On note $c \vdash_M^* c'$ où c et c' sont deux configurations de la machine de Turing M quand il existe une suite de configuration (c_i) telle que $c \vdash_M c_1 \vdash_M c_2 \vdash_M \dots \vdash_M c_{n-1} \vdash_M c_n \vdash_M c'$

3.2.4 Exemple

Considérons à nouveau la machine de Turing $M = (Q, q_0, \{q_f\}, \{1, B\}, B, \{1\}, \delta)$, où $Q = \{q_0, q_i, q_p, q_f\}$ et δ est définie par le tableau suivant :

δ	1	B
q_0	$(q_i, 1, D)$	(q_f, B, G)
q_i	$(q_p, 1, D)$	
q_p	$(q_i, 1, D)$	(q_f, B, G)

Le calcul de M sur 111 est : $q_0111 \vdash_M 1q_i11 \vdash_M 11q_p1 \vdash_M 111q_i$, il s'arrête alors dans l'état q_i qui est non-final. Il est illustré par la figure 3.6.

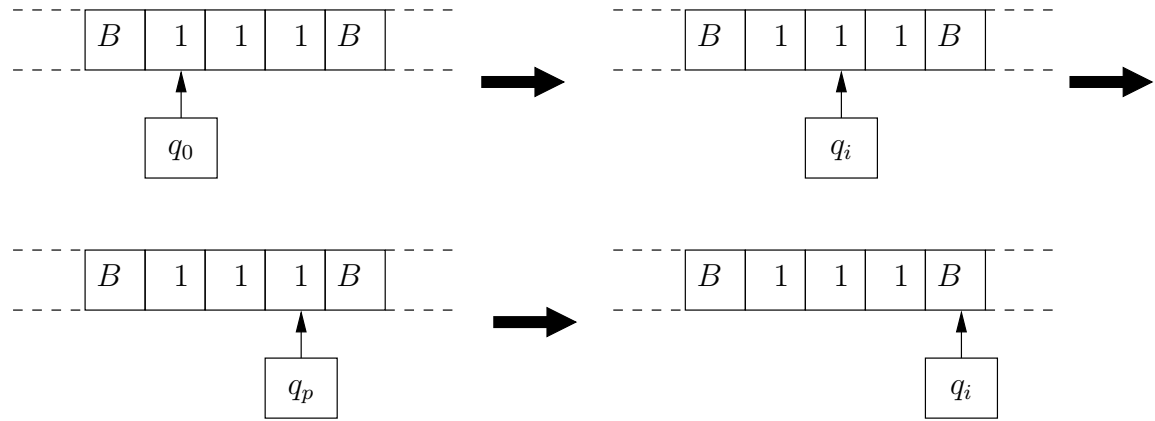


FIGURE 3.6 – Calcul de M sur 111.

Le calcul de M sur 1111 est : $q_01111 \vdash_M 1q_i111 \vdash_M 11q_p11 \vdash_M 111q_i1 \vdash_M 1111q_p \vdash_M 111q_f1$, il s'arrête alors dans l'état q_f qui est final. Il est illustré par la figure 3.7.

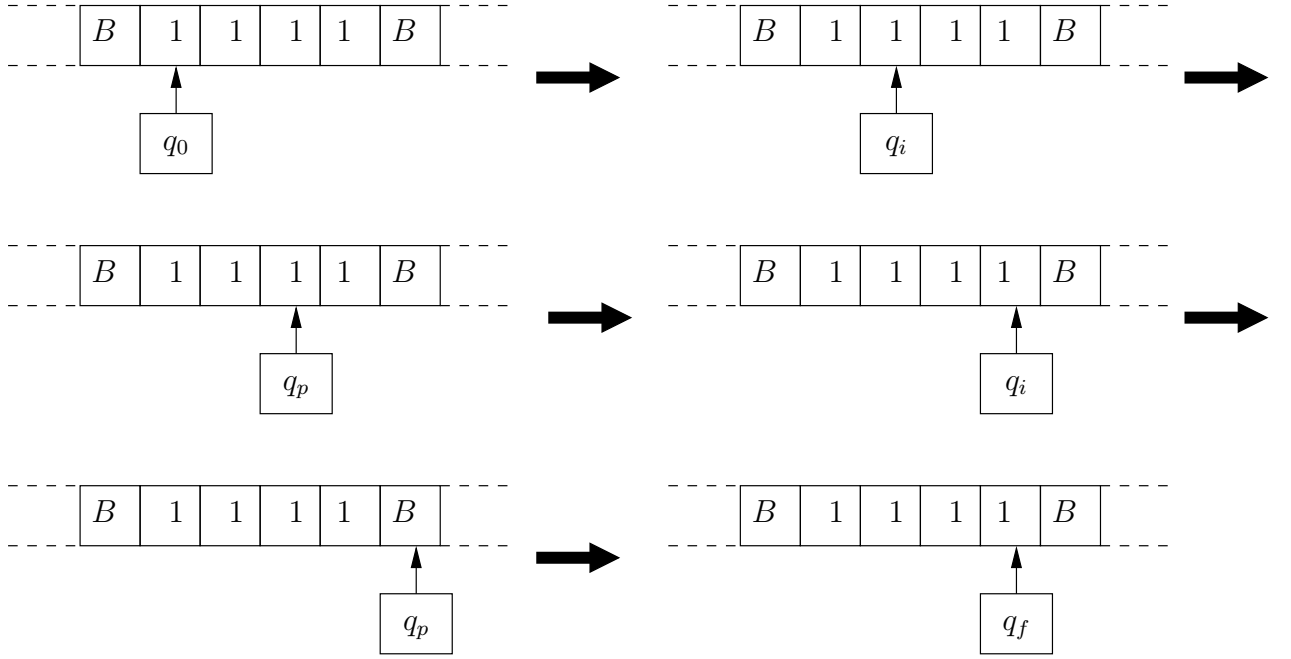


FIGURE 3.7 – Calcul de M sur 1111.

La machine est dans l'état de contrôle q_i si le nombre de 1 à gauche de sa tête de lecture est impair et dans l'état de contrôle q_p si le nombre de 1 à gauche de sa tête de lecture est pair. Enfin, elle est dans l'état q_f si elle pointe sur un 1, qu'il y a un nombre impair de 1 à sa gauche et un B à sa droite ou si elle pointe sur B et qu'il n'y a que des B sur le ruban.

Par conséquent, les calculs de M sur les mots de longueur paire s'achèvent dans l'état q_f qui est final alors que les calculs de M sur les mots de longueur impaire s'achèvent l'état q_i qui est non final.

3.2.5 Mots et langages reconnus

Étant donné une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ et un mot $\omega \in \Sigma^*$, trois situations peuvent se produire lors du calcul de M sur ω :

- le calcul s'arrête dans un état final, c'est-à-dire $q_0\omega \vdash_M^* \alpha q \beta$ avec $q \in F$. On dit alors que ω est **reconnu (accepté)** par M ;
- le calcul s'arrête dans un état non final et on dit que ω est **refusé** par M ;
- le calcul ne s'arrête pas.

Le langage **reconnu (accepté)** par une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ est $L(M) = \{\omega \mid \omega \in \Sigma^* \text{ et } q_0\omega \vdash_M^* \alpha q \beta \text{ avec } q \in F \text{ et } \alpha, \beta \in \Gamma^*\}$.

Exemple 8

La machine de Turing du paragraphe 3.2.4 reconnaît donc l'ensemble des mots de longueur paire sur l'alphabet $\{1\}$.

Par convention, on écrit toujours δ de façon à ce qu'elle ne soit jamais définie pour les éléments de F . Cela force M à s'arrêter aussitôt qu'une configuration est acceptée. Quand M est bien définie par le contexte, on remplace $\vdash_M$ par $\vdash$ et $\vdash_M^*$ par $\vdash^*$.

3.3 Langages et fonctions décidables

Le langage $L \subseteq \Sigma^*$ est **décidable** s'il existe une machine de Turing M telle que $L = L(M)$ et si M s'arrête sur tous les mots $\omega \in \Sigma^*$.

Le langage formé par les mots de longueur paire sur $\{1\}$ est décidable car il est reconnu par la machine de Turing du paragraphe 3.2.4 et le calcul de cette machine s'arrête pour tous les mots en donnant la bonne réponse.

Soit une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ et $f : \Sigma^* \rightarrow \Sigma^*$ une fonction. On dit que M **décide** f si M accepte tous les mots pour lesquels f est définie, refuse tous les mots pour lesquels f n'est pas définie et si, quand $f(\omega)$ est définie, $q_0\omega \vdash_M^* \alpha q\beta$ avec $q \in F$ et $f(\omega) = \alpha\beta$. Une fonction est **décidable** s'il existe une machine de Turing qui la décide.

Exemple 9

La machine de Turing $M = (Q, q_0, \{q_f\}, \{0, 1, B\}, B, \{0, 1\}, \delta)$, où $Q = \{q_0, q_r, q_f\}$ et δ est définie par le tableau suivant :

δ	0	1	B
q_0	$(q_0, 0, D)$	$(q_0, 1, D)$	(q_r, B, G)
q_r	$(q_f, 1, G)$	$(q_r, 0, G)$	$(q_f, 1, D)$

M décide la fonction qui ajoute 1 à un nombre écrit en binaire.

Lors d'un calcul, la machine reste dans l'état q_0 jusqu'à ce que la tête de lecture arrive à la droite du nombre, puis reste dans l'état q_r tant qu'il y a une retenue. Dès qu'il n'y plus de retenue le calcul s'arrête.

Voici le calcul sur 101 :

$$q_0101 \vdash 1q_001 \vdash 10q_01 \vdash 101q_0 \vdash 10q_r1 \vdash 1q_r00 \vdash q_f110$$

Voici le calcul sur 11 :

$$q_0 11 \vdash 1q_0 1 \vdash 11q_0 \vdash 1q_r 1 \vdash q_r 10 \vdash q_r B00 \vdash 1q_f 00$$

Chapitre 4

Variations sur la machine de Turing

Contents

4.1	Machine à plusieurs têtes et plusieurs rubans	20
4.1.1	Surplace	20
4.1.2	Plusieurs têtes et rubans	21
4.2	Machine non-déterministe	25
4.3	Machine à 3 symboles	26

Nous allons voir d'autres présentations des machines de Turing. Chacune d'entre elles est équivalente au modèle standard.

4.1 Machine à plusieurs têtes et plusieurs rubans

4.1.1 Surplace

On peut rajouter au modèle de la machine de Turing la possibilité de rester à la même position à chaque pas de calcul.

Il suffit de modifier la définition de la fonction de transition δ en ajoutant le symbole R pour le sur-place $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{G, D, R\}$.

Les conséquences pour la relation de transition $\vdash_M$ sur les configurations de M sont de rajouter l'ensemble $\{\alpha q a \beta \vdash_M \alpha q' b \beta \mid (q, a) \xrightarrow{\delta} (q', b, R), \alpha \in \Gamma^* \text{ et } \beta \in \Gamma^*\}$ (figure 4.1).

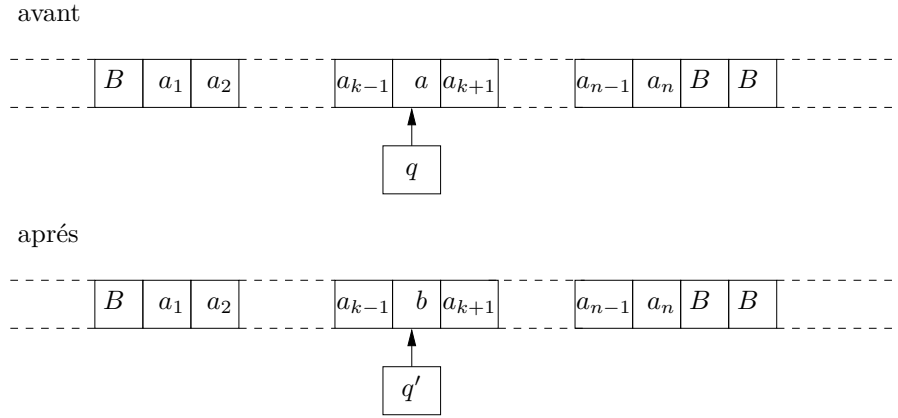


FIGURE 4.1 – Pas de calcul si $(q, a) \xrightarrow{\delta} (q', b, R)$.

Il est assez facile de simuler une telle machine par une machine de Turing standard en doublant tous les états q par des états q_{bis} , en remplaçant toute transition $(q, a) \xrightarrow{\delta} (q', b, R)$, par la transition $(q, a) \xrightarrow{\delta} (q'_{bis}, b, D)$ et en ajoutant l'ensemble de transitions $\{(q_{bis}, x) \xrightarrow{\delta} (q, x, G) \mid x \in \Gamma \text{ et } q \in Q\}$.

La possibilité de faire du surplace ne présente pas beaucoup d'intérêt pour une machine ayant une tête de lecture mais elle s'avère très utile dans le cas d'une machine à plusieurs têtes de lecture et plusieurs rubans.

4.1.2 Plusieurs têtes et rubans

On peut définir un modèle avec plusieurs bandes disjointes et une tête de lecture par ruban.

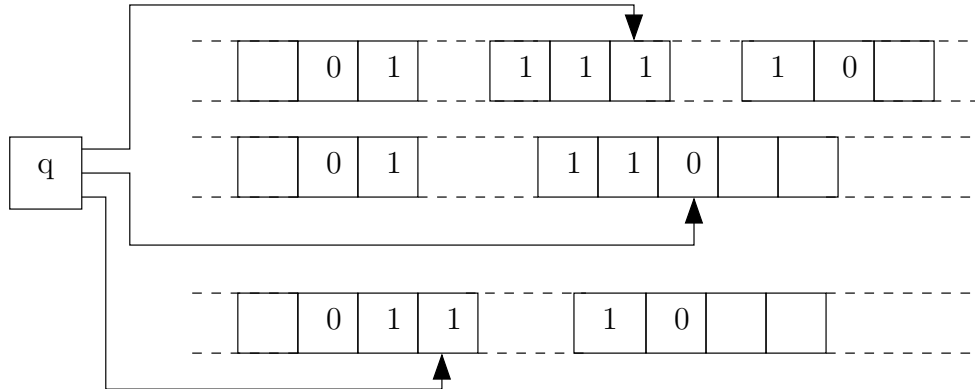


FIGURE 4.2 – configuration d'une machine à plusieurs rubans

Une configuration d'une telle machine est représentée par la figure 4.2.

À nouveau, la fonction de transition δ est la seule partie de la définition qui est modifiée. Dans une machine à plusieurs rubans et plusieurs têtes de lecture, $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{G, D, R\}^k$.

L'un des rubans contient le mot en donnée au début du calcul. Lors d'un pas de calcul, la machine change d'état, remplace le symbole qu'elle voit sur chaque ruban et déplace d'une case à gauche ou à droite ou laisse sur place chaque tête de lecture indépendamment l'une de l'autre. Les mouvements des têtes de lecture sont supposés simultanés.

Exemple 10

Nous allons voir une machine de Turing à deux rubans qui décide l'ensemble des palindromes sur l'alphabet $\{a, b\}$.

Un palindrome est un mot qui se lit de la même façon de gauche à droite et de droite à gauche. Formellement, on peut définir le miroir du mot $\omega = u_1 u_2 \cdots u_n$ par $\omega^r = u_n u_{n-1} \cdots u_2 u_1$. L'ensemble des palindromes sur l'alphabet A est $\{\omega \in A^* \mid \omega = \omega^r\}$.

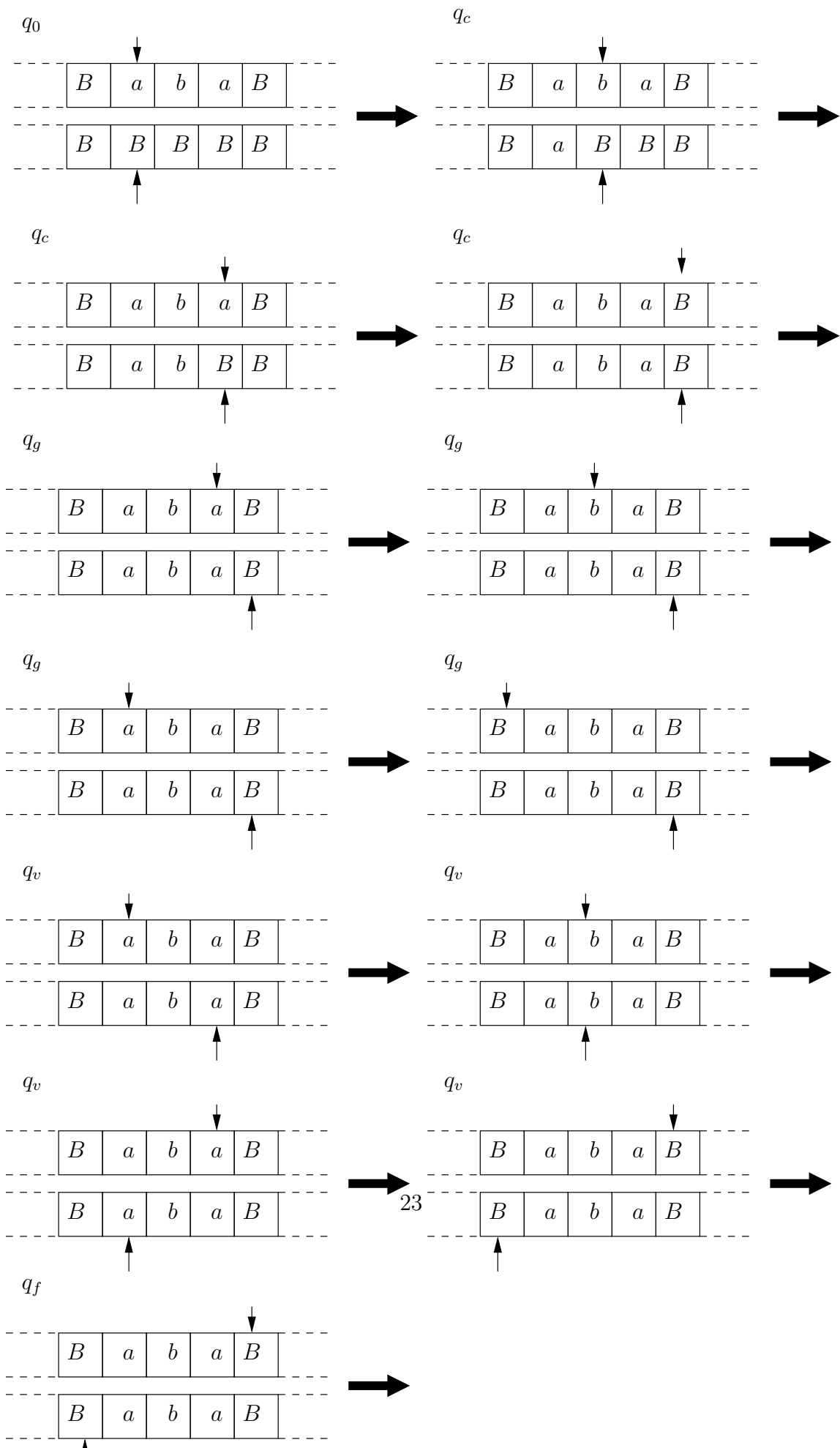
La machine de Turing $M = (Q, q_0, q_f, \{a, b, B\}, B, \{a, b\}, \delta)$, où $Q = \{q_0, q_c, q_g, q_v, q_f\}$ et δ est définie par le tableau suivant :

δ	a, a	b, b	B, B	a, B	b, B
q_0			(q_f, B, B, D, D)	(q_c, a, a, D, D)	(q_c, b, b, D, D)
q_c			(q_g, B, B, G, R)	(q_c, a, a, D, D)	(q_c, b, b, D, D)
q_g			(q_v, B, B, D, G)	(q_g, a, B, G, R)	(q_g, b, B, G, R)
q_v	(q_v, a, a, D, G)	(q_v, b, b, D, G)	(q_f, B, B, R, R)		

décide l'ensemble des palindromes sur l'alphabet $\{a, b\}$.

Lors d'un calcul, la machine commence par copier le mot qui est sur le premier ruban sur le second ; elle est alors dans l'état q_c . Puis, elle retourne au début du mot du premier ruban tout en restant à la fin du mot du deuxième ruban ; la machine est alors dans l'état q_g . Enfin, elle vérifie que les lettres coïncident en parcourant le mot sur le premier ruban par la droite et celui du second par la gauche ; elle est alors dans l'état q_v . Quand elle rencontre les deux caractères B , le mot est un palindrome.

Les figures 4.3 et 4.4 présentent le calcul pour le mot aba .



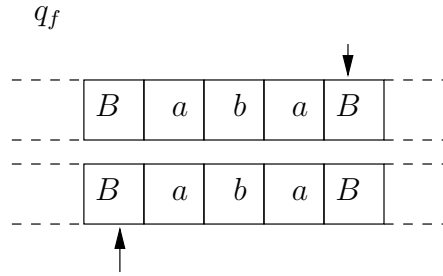


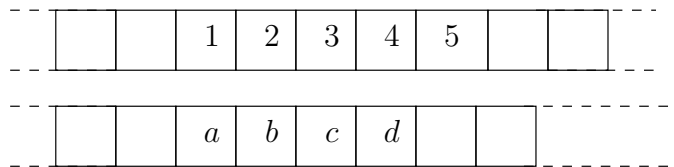
FIGURE 4.4 – calcul pour $aba(fin)$

Simuler une machine à plusieurs rubans avec une machine de Turing standard est un peu technique. Voici l'idée pour simuler une machine à deux rubans M avec une machine à un ruban M' . Tout d'abord, il faut que le ruban unique de M' contienne le contenu des deux rubans de M . Pour cela, on considère une case sur deux du ruban de M' et on y place le mot inscrit sur le premier ruban de M . Sur les cases laissées, on écrit le mot du second ruban.

Exemple 11

Simulation de 2 rubans avec un ruban :

rubans de M :



ruban de M' :

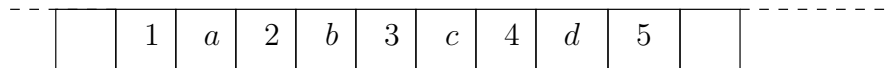


FIGURE 4.5 – Simulation d'une machine avec deux rubans.

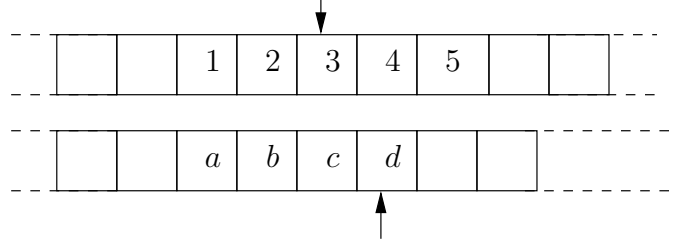
Ensuite, il faut indiquer les positions des têtes de lecture. Cela ne peut pas se faire par l'état de contrôle de M' car Q est fini alors que l'écart entre les deux têtes de lecture n'est pas borné a priori. Il faut donc coder ces positions sur le ruban. Pour cela, on fait une copie de l'alphabet Γ du ruban de M et pour chaque symbole a , on rajoute a' . Quand une case du ruban de M' contient a' , cela signifie que la tête de lecture de M pointe sur la case correspondante et qu'elle contient a . Quand

une case contient a , cela signifie que la case correspondante de M contient a mais que la tête de lecture ne la pointe pas.

Exemple 12

Simulation de 2 rubans avec un ruban (suite) :

rubans de M :



ruban de M' :

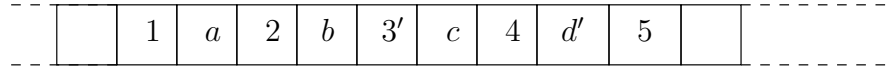


FIGURE 4.6 – Simulation d’une machine avec deux rubans.

Pour simuler une transition de M , M' commence par parcourir son ruban afin de connaître les symboles sur lesquels pointent M , puis il reparcourt son ruban pour modifier les valeurs des symboles et déplacer les “primes” afin d’indiquer les nouvelles places des têtes de lecture.

4.2 Machine non-déterministe

Une machine de Turing est **non-déterministe** si la fonction de transition δ est remplacée par une relation $\delta \subseteq Q \times \Gamma \times Q \times \Gamma \times \{G, D\}$. C’est-à-dire que pour $q \in Q$ et $a \in \Gamma$, il se peut que $\delta(q, a)$ ait plusieurs valeurs possibles (mais elles restent en nombre fini). Pour cette raison dans une configuration donnée $\alpha q \beta$, il peut exister plusieurs $\alpha' q' \beta'$ tels que $\alpha q \beta \vdash_M \alpha' q' \beta'$.

Le langage accepté par une machine de Turing non-déterministe est :

$$L(M) = \{\omega \mid \omega \in \Sigma^* \text{ et } q_0 \omega \vdash_M^* \alpha q \beta \text{ avec } q \in F \text{ et } \alpha, \beta \in \Gamma^*\}$$

Une machine de Turing non-déterministe accepte donc un mot ω s’il existe au moins un calcul qui mène $q_0 \omega$ à une configuration $\alpha q \beta$ où $q \in F$.

Afin de bien distinguer les deux définitions, une machine de Turing standard est appelée machine de Turing **déterministe**.

Théorème : il existe une machine non-déterministe M acceptant un langage L si et seulement s'il existe une machine déterministe M' acceptant L .

L'idée de la preuve de ce théorème est que l'ensemble des configurations de M pour une donnée ω peut être structuré comme un arbre dont la racine est la configuration initiale et les fils d'un nœud correspondant à une configuration c sont toutes les configurations qui découlent de c ; chaque branche correspond alors à un calcul possible de M pour la donnée ω et chaque branche se terminant par une configuration acceptée représente une suite de choix qui mène M à accepter ω . On peut alors construire une machine déterministe M' qui explore cet arbre en largeur en cherchant une branche qui se termine par une configuration acceptée.

4.3 Machine à 3 symboles

Pour toute machine de Turing M , il existe une machine de Turing $M' = (Q, q_0, F, \{0, 1, B\}, B, \{0, 1\}, \delta)$ qui reconnaît le même langage.

L'idée est de remplacer chaque symbole de l'alphabet de M par un nombre distinct fixé de 0 et de séparer les 0 des symboles de 2 cases consécutives du ruban de M par un 1.

Exemple 13

Si le ruban de M contient $abac$ et si a est codé par 0, b par 00 et c par 000, le ruban de M' contiendra 101001010001.

Chapitre 5

Décidabilité et indécidabilité

Contents

5.1	Thèse de Church	27
5.2	Langages et problèmes décidables	28
5.2.1	Propriétés des langages décidables	28
5.3	Problèmes indécidables	30
5.3.1	Codage d'une machine de Turing	30
5.3.2	Premier langage indécidable	31
5.4	Réductions et problèmes indécidables	32
5.4.1	Problème de l'acceptation ou problème universel	32
5.4.2	Problème de l'arrêt	32

Dans ce chapitre, on se demande quand est-ce qu'il existe un algorithme pour résoudre un problème? Nous allons commencer par voir que cela revient à se demander quand est-ce qu'il existe une machine de Turing qui décide ce problème.

Nous allons ensuite voir des problèmes indécidables et expliquer comment, à partir de ces problèmes, montrer que d'autres problèmes sont indécidables.

5.1 Thèse de Church

Selon la thèse de Church-Turing, **tout langage de programmation non trivial équivaut à une machine de Turing**. Il en résulte que tout programme que l'on peut écrire dans un langage pourrait être également écrit dans n'importe quel autre langage. La différence entre les langages ne réside donc pas dans ce qu'ils permettent de programmer, mais dans ce qui est facile ou commode de programmer avec chaque langage : cette différence est d'ordre non pas logique, mais pratique.

Il n'est pas possible de prouver formellement ce résultat, car il fait appel à la notion informelle d'algorithme. Par contre, pour chacune des notions d'algorithme

proposées jusqu'à aujourd'hui (algorithme = programme C, programme Java, ou Programme de machine RAM), on a montré que pour tout algorithme, il existe une machine de Turing qui le décide.

Cette thèse explique l'importance des machines de Turing. En effet, il existe un algorithme pour résoudre un problème si et seulement s'il existe une machine de Turing qui le décide. Le modèle des machines de Turing est très simple : il permet de faire des preuves, notamment quand on veut prouver qu'un problème n'a pas de solution algorithmique.

5.2 Langages et problèmes décidables

Le langage $L \subseteq \Sigma^*$ est **décidable** s'il existe une machine de Turing M telle que $L = L(M)$ (i.e., le langage L est le langage reconnu par la machine de Turing M) et si M s'arrête sur tous les mots $\omega \in \Sigma^*$.

Un problème est dit **décidable** s'il existe une machine de Turing qui le décide, c'est-à-dire s'il existe une machine de Turing qui s'arrête sur toutes les instances du problème en donnant la bonne réponse.

5.2.1 Propriétés des langages décidables

Le complémentaire d'un langage décidable est décidable.

Preuve : Soit L un langage décidable et soit $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ une machine de Turing qui décide L . On va construire une machine de Turing $M' = (Q', q'_0, F', \Gamma', B', \Sigma', \delta')$ qui décide le complémentaire de L que l'on note $\bar{L}$.

- $Q' = Q \cup \{q_n\}$, on utilise les mêmes états que ceux de M auxquels on rajoute un nouvel état q_n .
- $q'_0 = q_0$, on garde le même état initial.
- $F' = \{q_n\}$, l'ensemble des états finaux se réduit au nouvel état. En particulier, les états de F ne sont pas finaux pour M' .
- $\Gamma' = \Gamma$, $\Sigma' = \Sigma$, $B' = B$ on ne change ni l'alphabet du ruban, ni celui du langage, ni le symbole vide.
- δ' va contenir tout δ à laquelle on rajoute les transitions suivantes :
si δ n'est pas définie pour (q, a) et $q \notin F$, alors $(q, a) \xrightarrow{\delta} (q_n, a, G)$.
Autrement dit, là où M s'arrête dans un état non final, on rajoute une transition afin que M' s'arrête dans un état final.

Exemple 14

Considérons à nouveau la machine de Turing $M = (Q, q_0, q_f, \{1, B\}, B, \{1\}, \delta)$ du deuxième chapitre, où $Q = \{q_0, q_i, q_p, q_f\}$ et δ est définie par le tableau suivant :

δ	1	B
q_0	$(q_i, 1, D)$	(q_f, B, G)
q_i	$(q_p, 1, D)$	
q_p	$(q_i, 1, D)$	(q_f, B, G)

Elle reconnaît les mots contenant un nombre pair de 1.

M' vaut $(Q \cup \{q_n\}, q_0, \{q_n\}, \{1, B\}, B, \{1\}, \delta')$ et δ' est définie par le tableau suivant :

δ'	1	B
q_0	$(q_i, 1, D)$	(q_f, B, G)
q_i	$(q_p, 1, D)$	(q_n, B, G)
q_p	$(q_i, 1, D)$	(q_f, B, G)

Le calcul de 1111 sur M' est le même que pour M , $q_01111 \vdash_M 1q_i111 \vdash_M 11q_p11 \vdash_M 111q_i1 \vdash_M 1111q_p \vdash_M 111q_f1$, q_f n'est pas final pour M' donc 1111 n'est pas reconnu par M' .

Par contre le calcul de 111 devient $q_0111 \vdash_M 1q_i11 \vdash_M 11q_p1 \vdash_M 111q_iB \vdash_M 11q_n1$ étant donné l'ajout fait à δ . Or q_n est final pour M' donc M' reconnaît 111.

Fin de la preuve : Si un mot ω est reconnu par M , alors il existe un calcul de M pour ce mot qui s'arrête sur un état de F . Ce calcul existe aussi pour M' , il va également s'arrêter sur cet état de F car on n'a pas rajouté de transition partant d'un état de F . Cependant, comme cet état n'est pas final pour M' , M' ne reconnaît pas ω . Il n'existe pas d'autre calcul possible car δ' est une fonction.

Si un mot ω n'est pas reconnu par M , alors il existe un calcul de M pour ce mot qui s'arrête sur un état non final, donc n'appartenant pas à F . Ce calcul existe aussi pour M' et il se prolonge d'une transition qui l'amène dans l'état q_n étant donné les ajouts fait à δ . Par conséquent, M' reconnaît ω .

On a donc montré que ω est reconnu par M si et seulement s'il n'est pas reconnu par M' . Donc $L(M') = \bar{L}$. De plus, les calculs de M' prolongent ceux de M d'autant plus un pas car aucune transition ne part de l'état q_n , donc M' s'arrête bien sur tous les mots de Σ^n . Il s'ensuit que M' décide $\bar{L}$.

L'intersection de deux langages décidables est décidable.

Idée de la preuve : Soient L_1 et L_2 deux langages décidables, décidés respectivement par les machines de Turing M_1 et M_2 . L'idée consiste à copier le mot en donnée sur un ruban supplémentaire, à commencer le calcul sur M_1 . Si on arrive dans un état final pour M_1 , alors on commence le calcul sur la copie avec M_2 et le mot est accepté si et seulement s'il est accepté par M_2 . Sinon (si on arrive pas dans un état final pour M_1), on arrête le calcul et le mot est refusé.

L'union de deux langages décidables est décidable.

Idée de la preuve : Soient L_1 et L_2 deux langages décidables, décidés respectivement par les machines de Turing M_1 et M_2 . L'idée consiste à copier le mot en donnée sur un ruban supplémentaire et à mener en parallèle le calcul du mot par M_1 sur le premier ruban et celui du mot par M_2 sur le ruban supplémentaire. Quand l'un des calculs s'arrête, si l'état rencontré est final alors le mot est reconnu, sinon on poursuit le calcul inachevé en restant sur place pour le calcul achevé et le mot est accepté si et seulement si le calcul inachevé aboutit à un état final.

5.3 Problèmes indécidables

Un problème est dit **indécidable** quand il n'existe pas de machine de Turing qui le décide, c'est-à-dire qu'il n'existe pas de machine de Turing qui s'arrête sur toutes les instances du problème en donnant la bonne réponse. Cela signifie encore que pour toute machine de Turing M , il existe une instance du problème sur laquelle

- soit le calcul ne s'arrête pas
- soit le calcul s'arrête mais pas avec le bon résultat (état acceptant alors que l'instance doit être refusée, état non-acceptant alors que l'instance doit être acceptée ou mauvais mot sur le ruban qui doit contenir le résultat du calcul).

5.3.1 Codage d'une machine de Turing

On peut décrire les machines de Turing sous forme de mots sur un alphabet donné. Ces machines pourront être alors elles-mêmes des données pour d'autres machines.

À la fin du chapitre "Variations sur la machine de Turing", on a vu que pour toute machine de Turing M , il existe une machine de $M' = (Q, q_0, F, \{0, 1, B\}, B, \{0, 1\}, \delta)$ qui reconnaît le même langage. On peut même avoir $F = \{q_1\}$ et $Q = \{q_0, q_1, \dots, q_n\}$. M' est alors totalement spécifiée par δ qui peut être écrite sous forme de mot.

Exemple 15

La machine de Turing $M = (Q, q_0, q_1, \{1, B\}, B, \{1\}, \delta)$, où $Q = \{q_0, q_i, q_p, q_1\}$ et δ est définie par le tableau suivant :

δ	1	B
q_0	$(q_i, 1, D)$	(q_1, B, G)
q_i	$(q_p, 1, D)$	
q_p	$(q_i, 1, D)$	(q_1, B, G)

qui reconnaît les mots de longueur paire sur l'alphabet $\{1\}$ est totalement spécifiée par le mot : $\delta(q_0, 1) = (q_i, 1, D); \delta(q_0, B) = (q_1, B, G); \delta(q_i, 1) = (q_p, 1, D); \delta(q_p, 1) = (q_i, 1, D); \delta(q_p, B) = (q_1, B, G)$

Il est possible de coder ce mot sur l'alphabet $\{0, 1\}^*$, en remplaçant chaque symbole par un nombre donné de 0 et en les séparant par des 1. Étant donnée une machine M , on note $\langle M \rangle$ son codage sur $\{0, 1\}^*$.

Par exemple, si on choisit le codage suivant :

0	0
1	00
B	000
G	0000
D	00000
q_0	0000000
q_1	00000000
q_i	000000000
q_p	0000000000

$\delta(q_0, 1) = (q_i, 1, D)$ est codé par 10000001001000000001001000001 et M est codé par 10000001001000000001001000001100000010001000000010001000011...

5.3.2 Premier langage indécidable

Supposons que l'on énumère tous les mots sur $\{0, 1\}^*$ en commençant par les plus courts, puis en ordre lexicographique pour chaque longueur et que l'on nomme ω_i le i -ème mot dans l'énumération. On associe à tout entier binaire j , une machine de Turing M_j :

- si ω_j correspond au codage d'une machine de Turing, on nomme celle-ci M_j
- sinon M_j est la machine qui reconnaît le langage vide.

On considère le langage suivant : $L_d = \{\omega_i \mid \omega_i \notin L(M_i)\}$. L_d est indécidable.

Supposons que L_d soit décidable, il existe alors une machine de Turing M qui décide L_d . Il existe donc un indice j_0 (même plusieurs) tel que $M = M_{j_0}$. Est-ce que $\omega_{j_0} \in L(M_{j_0})$?

$$\omega_{j_0} \in L(M_{j_0}) \Leftrightarrow \omega_{j_0} \in L_d \Leftrightarrow \omega_{j_0} \notin L(M_{j_0})$$

Cette contradiction permet de conclure qu'il n'existe pas de machine de Turing M qui décide L_d .

5.4 Réductions et problèmes indécidables

L'idée d'une réduction est de montrer qu'un langage L n'est pas décidable en utilisant le fait qu'un autre langage L' ne l'est pas. Pour le prouver, on montre que s'il existait une machine M telle que $L = L(M)$ alors, il existerait une machine M' telle que $L' = L(M')$. En sachant que L' n'est pas décidable, on en déduit par contraposition que M ne peut exister.

5.4.1 Problème de l'acceptation ou problème universel

Problème universel ou de l'acceptation

Données : Une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ et $\omega \in \Sigma^*$

Question : Est ce que M accepte ω ?

Le problème de l'acceptation est indécidable.

Soit $L_u = \{(M, \omega) \mid \omega \in L(M)\}$. On va en fait montrer que le complémentaire de L_u , noté $\overline{L_u}$ est indécidable et on conclura en utilisant la propriété que le complémentaire d'un langage décidable est décidable.

On va réduire L_d à $\overline{L_u}$. Supposons qu'il existe une machine M pouvant décider $\overline{L_u}$. On construit une machine M' qui décide L_d . Sur la donnée ω , M' trouve un i tel que $\omega = \omega_i$ (il suffit pour cela d'énumérer les mots et de s'arrêter quand on trouve ω). Avec ce i , M' construit la donnée M_i, w_i et la donne à M . Si M accepte M_i, w_i , alors M' accepte w_i . Sinon, elle rejette.

Il s'ensuit que M' accepte w_i si et seulement si M_i n'accepte pas ω_i , donc $w_i \in L_d$. Mais $\omega = \omega_i$, donc M' accepte exactement les mots de L_d . Puisque M' ne peut pas exister, M ne peut pas exister non plus.

5.4.2 Problème de l'arrêt

Problème de l'arrêt

Données : Une machine de Turing $M = (Q, q_0, F, \Gamma, B, \Sigma, \delta)$ et $\omega \in \Sigma^*$

Question : Est ce que M s'arrête sur ω ?

Le problème de l'arrêt est indécidable.

Soit $L_{arr} = \{(M, \omega) \mid M \text{ s'arrête sur } \omega\}$. On va montrer que L_{arr} se réduit à L_u .

Supposons qu'il existe une machine de Turing M_{arr} qui décide L_{arr} . On va construire une machine qui décide L_u . Sur la donnée M, ω , M' passe le contrôle à M_{arr} . Si cette dernière s'arrête sur M, ω en acceptant, alors on simule M sur ω

et on accepte ω si et seulement si M accepte ω . Si, par contre M_{arr} rejette M, ω , alors on rejette M, ω .

M' décide bien L_u . Puisque M' ne peut exister, M_{arr} ne peut pas exister non plus.

Chapitre 6

Classes de complexité

Contents

6.1	Définitions de la complexité d'une machine de Turing	35
6.2	Définitions des classes de complexité	35
6.2.1	Comparaison de fonctions	36
6.2.2	Classe de complexité d'un problème	36
6.2.3	Propriété	37
6.2.4	Problèmes traitables	37
6.3	Thèse de l'invariance	37

On a vu dans les chapitres précédents qu'il existe deux types de problèmes : ceux qui ont une solution algorithmique et ceux qui n'en ont pas comme le problème de Post par exemple. On a vu, grâce à la thèse de Church, que déterminer à quel type un problème appartient revient à déterminer s'il existe une machine de Turing qui le résout.

Nous allons maintenant nous intéresser aux problèmes décidables (ayant une solution algorithmique) dans le but de distinguer ceux qui ont une solution algorithmique satisfaisante des autres, c'est-à-dire ceux qui permettent d'obtenir la solution en un temps acceptable et avec une mémoire raisonnable. Dans ce chapitre, nous allons voir comment classer les problèmes en fonction du temps et de la mémoire qu'ils nécessitent pour être résolus.

Dans tous les chapitres qui suivent, on ne considère que les fonctions sur les entiers f telles que $f(n+1) \geq f(n), \forall n \in \mathbb{N}$. On impose donc que la suite $(f(n))_{n \in \mathbb{N}}$ soit croissante.

6.1 Définitions de la complexité d'une machine de Turing

On va commencer par associer deux fonctions C_t et C_e à une machine de Turing, qu'elle soit déterministe ou non-déterministe.

- C_t est la **complexité en temps**.
- C_e est la **complexité en espace**.

Dans le cas d'une machine déterministe, on commence par compter le nombre d'étapes de calcul $c_t(\omega)$ et le nombre de cases du ruban utilisées $c_e(\omega)$ nécessaires au passage de la configuration initiale associée à une donnée ω à la configuration terminale associée au résultat (oui ou non pour un problème de décision, une valeur $f(\omega)$ pour une fonction).

Dans le cas d'une machine non-déterministe, on note $c_t(\omega)$ le nombre d'étapes pour le calcul le plus court et $c_e(\omega)$ le nombre de cases du ruban utilisées pour le calcul le moins gourmand en ressources permettant de passer de la configuration initiale associée à une donnée ω à la configuration terminale associée au résultat.

On regroupe alors les données ω en fonction de leur taille, c'est-à-dire du nombre de cases du ruban utilisées pour écrire ω , notée $|\omega|$. On remarque d'ailleurs que $c_t(\omega)$ et $c_e(\omega)$ dépendent souvent uniquement de $|\omega|$.

On définit alors $C_t(n)$ comme le nombre maximum de pas de calcul pour obtenir le résultat sur une donnée de taille n . Autrement dit :

$$C_t(n) = \sup\{c_t(\omega) : |\omega| = n\}$$

et $C_e(n)$ comme le nombre maximum de cases de ruban pour obtenir le résultat sur une donnée de taille n . Autrement dit :

$$C_e(n) = \sup\{c_e(\omega) : |\omega| = n\}$$

6.2 Définitions des classes de complexité

Les fonctions $C_t(n)$ et $C_e(n)$ obtenues précédemment peuvent avoir des allures très complexes qui ne seront pas très pratiques à utiliser pour comparer deux machines de Turing résolvant le même problème. On va donc les associer à des fonctions mieux connues comme les fonctions linéaires (de la forme $n \rightarrow an$), polynomiales (de la forme $n \rightarrow n^c$) ou exponentielles (de la forme $n \rightarrow c^n$). Pour cela, on s'intéresse à la notion de domination asymptotique.

6.2.1 Comparaison de fonctions

Soient f et g deux fonctions de $\mathbb{N}$ dans $\mathbb{R}^+$. f est **dominée asymptotiquement** par g , $f = O(g)$ si et seulement si $\exists c \in \mathbb{R}^{+*}$, $\exists n_0 \in \mathbb{N}$ tels que $\forall n > n_0$, $f(n) \leq cg(n)$. Autrement dit, il existe un réel strictement positif c tel que pour n assez grand, $f(n)$ est plus petit que $cg(n)$. Cette situation est illustrée par la figure 6.1.

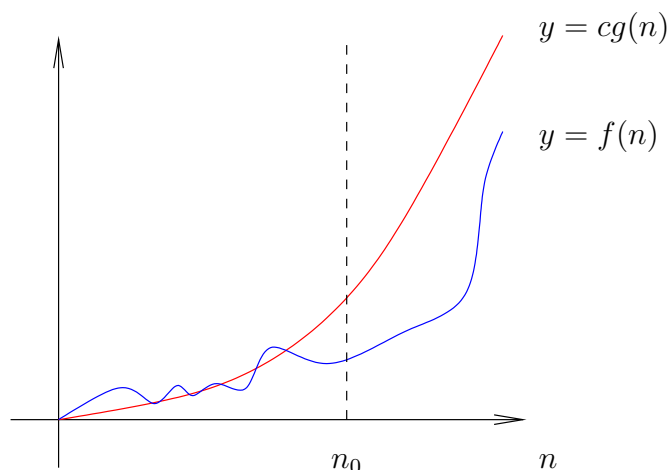


FIGURE 6.1 – $f = O(g)$

L'idée est que $f(n)$ ne dépasse pas $g(n)$ fois une certaine constante pour les grandes valeurs de n .

Exemple 16

$n + 4n^2 + 5n^5 = O(n^5)$, $4 \log(n) = O(n)$.

6.2.2 Classe de complexité d'un problème

Étant donné un problème P décidable, on s'intéresse aux machines de Turing déterministes et non-déterministes à un ruban qui le résolvent, ainsi qu'au nombre d'étapes de calcul et au nombre de cases du ruban utilisées pour passer des données initiales à la réponse finale. On ne va pas retenir les fonctions C_t et C_e associées à ces machines de Turing mais les fonctions "simples" qui les dominent asymptotiquement. On regarde uniquement ce qui se passe pour les grandes valeurs de n car ce sont celles qui peuvent poser problème. En effet, c'est avec elles que la mémoire peut être insuffisante ou le temps de calcul trop important.

Soit f une fonction de $\mathbb{N}$ dans $\mathbb{R}^+$ "simple". On définit quatre familles de classes de complexité :

- $\text{DTime}(f)$:
 P appartient à la classe $\text{DTime}(f)$ s'il existe une machine de Turing déterministe M telle que $C_t(n) = O(f(n))$.
- $\text{DSpace}(f)$:
 P appartient à la classe $\text{DSpace}(f)$ s'il existe une machine de Turing déterministe M telle que $C_e(n) = O(f(n))$.
- $\text{NTime}(f)$:
 P appartient à la classe $\text{NTime}(f)$ s'il existe une machine de Turing non-déterministe M telle que $C_t(n) = O(f(n))$.
- $\text{NSpace}(f)$:
 P appartient à la classe $\text{NSpace}(f)$ s'il existe une machine de Turing non-déterministe M telle que $C_e(n) = O(f(n))$.

6.2.3 Propriété

On a la propriété suivante :

$$\text{DTime}(f(n)) \subseteq \text{NTime}(f(n)) \subseteq \bigcup_{c \geq 1} \text{DTime}(c^{f(n)})$$

6.2.4 Problèmes traitables

Nous nous intéresserons tout particulièrement à deux classes :

- $\mathcal{P} = \bigcup_{c \in \mathbb{R}^+} \text{DTime}(n^c)$
- $\mathcal{NP} = \bigcup_{c \in \mathbb{R}^+} \text{NTime}(n^c)$

Par convention, les problèmes qui appartiennent à la classe $\mathcal{P}$ sont dit **polynomiaux** et sont considérés comme **traitables** (utilisables ou *tractable*), c'est-à-dire qu'ils possèdent une solution algorithmique qui ne prend ni trop de temps, ni trop d'espace.

6.3 Thèse de l'invariance

La traduction d'un programme en une machine de Turing déterministe introduit au plus un ralentissement quadratique (on élève la fonction au carré), et un facteur constant sur l'espace. La classe $\mathcal{P}$ est stable quel que soit le modèle choisi.

Chapitre 7

Comparaisons d'algorithmes

Contents

7.1	Comparaison de fonctions	38
7.2	Complexité d'un algorithme	39
7.2.1	Opérations fondamentales	39
7.2.2	Nombre d'opérations	40
7.2.3	Complexité en moyenne et dans le pire des cas	44

Dans le chapitre précédent, on a vu comment classer différents problèmes en fonction du temps et de l'espace que leur résolution prend. Quand on a deux algorithmes pour résoudre le même problème, on peut les comparer en déterminant quelles sont leur complexité. Cependant, ce n'est pas très pratique de les traduire dans le modèle des machines de Turing. Dans ce chapitre, nous allons voir concrètement comment les évaluer dans le but de les comparer.

7.1 Comparaison de fonctions

Soient f et g deux fonctions de $\mathbb{N}$ dans $\mathbb{R}^+$. La notation $f = O(g)$ a été définie au chapitre précédent. Nous allons voir une comparaison plus fine :

Les fonctions f et g sont du **même ordre de grandeur asymptotique**, $f = \Theta(g)$, si et seulement si $\exists c_1, c_2 \in \mathbb{R}^{+*}$, $\exists n_0 \in \mathbb{N}$ tels que $\forall n > n_0$, $c_2 g(n) \leq f(n) \leq c_1 g(n)$. Autrement dit, il existe deux réels strictement positif c_1 et c_2 tels que pour n assez grand, $f(n)$ est compris entre $c_1 g(n)$ et $c_2 g(n)$. Cette situation est illustrée par la figure 7.1.

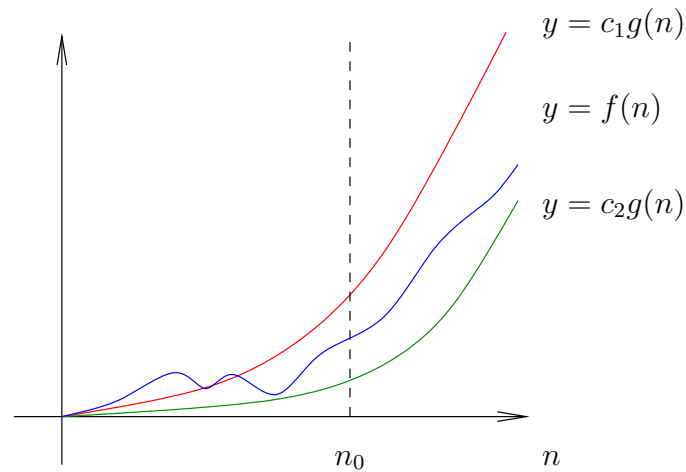


FIGURE 7.1 – $f = \Theta(g)$

$f = \Theta(g)$ est plus précis que $f = O(g)$ car dans le premier cas, on encadre le comportement de f (pour n très grand) tandis que dans le second, on la majore seulement. Lorsqu'on étudie une fonction f , on cherchera donc une fonction simple g telle que $f = \Theta(g)$. A défaut, on se contentera d'une fonction h telle que $f = O(h)$.

On peut faire l'analogie suivante : si $f = \Theta(g)$ équivaut à $a = b$, alors $f = O(h)$ équivaut à $a < c$.

7.2 Complexité d'un algorithme

On cherche à déterminer une mesure qui rende compte de la complexité intrinsèque des algorithmes, indépendamment de l'implémentation et qui permette ainsi de comparer entre eux des algorithmes. On va se consacrer uniquement à la complexité en temps.

7.2.1 Opérations fondamentales

Pour un problème donné, on recherche une ou plusieurs **opérations fondamentales** dans le sens où le temps d'exécution d'un algorithme résolvant ce problème est toujours proportionnel au nombre de ces opérations. Il est alors possible de comparer des algorithmes traitant ce problème selon cette mesure simplifiée.

Voici quelques exemples d'opérations fondamentales :

- lors de la recherche d'un élément dans un tableau, le nombre de comparaisons entre cet élément et les éléments du tableau.
- pour additionner deux matrices, le nombre d'additions.

7.2.2 Nombre d'opérations

Après avoir déterminé les opérations fondamentales, on compte le nombre d'opérations de chaque type. On note $nb(I)$, le nombre d'opérations des instructions I .

Voici quelques règles pour le déterminer :

7.2.2.1 Séquence

Lorsque les opérations sont dans une séquence d'instructions, leurs nombres s'ajoutent.

Exemple 17

```
variables a, b, c: entiers
début
  a <- 2;
  b <- 5 + a;
  c <- a + b + 2;
  a <- c;
fin
```

Dans ce programme, on peut considérer deux opérations fondamentales : l'affectation et la somme d'entiers. Nous allons regarder le nombre d'opérations fondamentales dans chacun des cas.

Pour l'affectation, il y en a une par instruction : une pour $a <- 2$; , une pour $b <- 5 + a$; , une pour $c <- a + b + 2$; et enfin une pour $a <- c$; . Ces instructions forment une séquence, donc on somme le nombre d'affectations pour chaque instruction, soit un total de quatre. Il y a donc quatre affectations pour ce programme.

Pour la somme, le nombre d'additions varie suivant les instructions : aucune pour $a <- 2$; , une pour $b <- 5 + a$; , deux pour $c <- a + b + 2$; et enfin aucune pour $a <- c$; . Ces instructions forment une séquence, donc on somme le nombre d'additions pour chaque instruction, soit un total de trois. Il y a donc trois additions pour ce programme.

7.2.2.2 Branchement conditionnel

$$nb(\text{si } C \text{ alors } I \text{ sinon } I') \leq nb(C) + \max\{nb(I), nb(I')\}.$$

Exemple 18

```
si ((a+b+c) < 10)
  alors b <- 5;
  sinon c <- a + b + 2;
```

```
fin si
```

*On considère comme opération fondamentale l'addition. Il y en a deux dans la conditionnelle ((a+b+c) < 10). Aucune dans le **alors** (b <- 5). Deux dans le **sinon** (c <- a + b + 2). Donc si la conditionnelle est satisfaite, il y a deux additions. Si elle n'est pas satisfaite, il y a quatre additions. Conclusion, le nombre d'additions pour cette instruction est majoré par quatre.*

7.2.2.3 Boucles

Le nombre d'opérations dans la boucle est la somme du nombre d'opérations pour chaque passage dans la boucle.

Exemple 19

```
pour i variant de 1 à 4 faire
  x <- x+i;
fin pour
```

*On considère comme opération fondamentale l'addition. À chaque passage dans la boucle, on effectue l'instruction **x <- x+i**; qui nécessite une addition. On passe dans la boucle quatre fois (une pour chaque valeur de i entre 1 et 4). On effectue donc une addition quand i vaut 1, une quand i vaut 2, une quand i vaut 3 et une quand i vaut 4. Cela fait un total de 4 additions.*

Exemple 20

```
pour i variant de 1 à n faire
  x <- x+i+3;
fin pour
```

*On considère toujours comme opération fondamentale l'addition. À chaque passage dans la boucle, on effectue l'instruction **x <- x+i+3**; qui nécessite deux additions. On passe dans la boucle n fois (une pour chaque valeur de i entre 1 et n). On effectue donc deux additions n fois, soit 2n additions.*

Exemple 21

```
pour i variant de 1 à 4 faire
  pour j variant de 1 à 3 faire
    x <- x+j;
  fin pour
fin pour
```

On considère comme opération fondamentale l'addition.

On regarde d'abord la boucle la plus interne :

```
pour j variant de 1 à 3 faire
  x <- x+j;
fin pour
```

Pour chaque valeur de j , on effectue l'instruction $x \leftarrow x+j$; qui nécessite une addition. Il y a trois valeurs pour j (1,2,3). On effectue donc une addition trois fois, soit trois additions au total par exécution de la boucle la plus interne.

On exécute la boucle la plus interne pour chaque valeur de i . Il y a quatre valeurs pour i . On effectue donc trois additions quatre fois, soit douze fois pour la totalité de l'instruction.

Exemple 22

Voici un programme qui ajoute deux matrices de taille $n \times m$.

```
variables A, B, C: tableaux de n x m réels
      i, j: entiers
début
  pour i variant de 1 à n faire
    pour j variant de 1 à m faire
      C[i,j] <- A[i,j]+B[i,j]
    fin pour
  fin pour
fin
```

Pour chaque valeur de j , il y a une addition. Il y a m valeurs pour j par exécution de la boucle la plus interne, donc m additions par exécution de la boucle la plus interne.

On exécute la boucle la plus interne pour chaque valeur de i . Or, il y a n valeurs pour i dans la boucle externe, cela fait en tout mn additions.

7.2.2.4 Appels de procédure ou de fonction

Lors d'appels à une procédure ou à une fonction, s'il n'y a pas de récursivité, on ajoute le nombre d'opérations pour chaque appel, sinon on doit souvent résoudre une relation de récurrence.

Exemple 23

```
fonction f (a: entier, b: entier): entier
début
  retourner a+b+2;
fin

début
variables x, y, z: entiers
  x <- f(2,3);
  z <- x+y;
  y <- f(4,x)+f(3,-2);
```

fin

On considère la fonction f , elle est composée d'une seule instruction **retourner** $a+b+2$; qui comprend deux additions. Donc, chaque appel à f entraîne l'exécution de deux additions.

Dans le programme principal, il y a trois instructions : $x \leftarrow f(2,3)$; $z \leftarrow x+y$; et $y \leftarrow f(4,x)+f(3,-2)$;

La première est composée d'un appel à f , donc elle nécessite deux additions.

La deuxième comprend une addition.

La troisième est composée de deux appels à f et d'une addition. Chaque appel à f demande deux additions. Donc, les deux appels à f nécessitent quatre additions. L'exécution de la troisième instructions demande donc cinq additions.

Le programme principal comprend donc en tout huit additions.

Exemple 24

Voici la fonction factorielle.

```
fonction factorielle (n: entier): entier
début
  si (n = 0)
    alors retourner 1;
    sinon retourner n * factorielle(n-1);
fin
```

L'opération fondamentale est la multiplication d'entiers. Soit $nb(n)$ le nombre de multiplication pour la donnée n , elle vérifie :

- $nb(0) = 0$
- $nb(n) = 1 + nb(n-1)$.

La démonstration par récurrence suivante permet de conclure que le nombre de multiplications pour calculer **factorielle** (n) est n .

Démonstration :

On considère comme hypothèse de récurrence sur n

$$H(n) : nb(n) = n.$$

L'hypothèse de récurrence est vraie pour $n=0$: $nb(0) = 0$.

On considère un entier n et on suppose que $H(n)$ est vraie. On va montrer que $H(n+1)$ est vraie. On sait que $nb(n+1) = 1 + nb(n)$ et par hypothèse, $nb(n) = n$. On obtient que $nb(n+1) = 1+n$ qui est précisément $H(n+1)$.

On en conclut que $nb(n) = n$ pour tout n .

Exemple 25

```
fonction f (n: entier): entier
```

```

début
  si ((n = 0) ou (n = 1))
    alors retourner 1;
    sinon retourner n + f(n-2);
fin

```

L'opération fondamentale est l'addition d'entiers. Soit $nb(n)$ le nombre d'additions pour la donnée n . Elle vérifie :

- $nb(0) = 0, nb(1) = 0$
- $nb(n) = 1 + nb(n-2)$.

Si on regarde les premières valeurs de $nb(n)$ (0, 0, 1, 1, 2, 2, etc), on peut penser que $nb(n) = \frac{n}{2}$. On va le montrer par récurrence :

démonstration :

On considère comme hypothèse de récurrence sur n

$$H(n) : nb(n) = \frac{n}{2} \text{ et } nb(n+1) = \frac{n+1}{2}.$$

L'hypothèse de récurrence est vraie pour $n=0$: $nb(0) = nb(1) = 0$.

On considère un entier n et on suppose que $H(n)$ est vraie. On va montrer que $H(n+1)$ est vraie.

Par hypothèse, on a déjà : $nb(n+1) = \frac{n+1}{2}$

On sait que $nb(n+2) = 1 + nb(n)$.

- si n est pair, $n+1$ est impair et $n+2$ est pair. Donc $\frac{n}{2} = \frac{n+1}{2}$ et $\frac{n+2}{2} = \frac{n+1}{2} + 1$.

Par hypothèse de récurrence, $nb(n) = \frac{n}{2}$, donc $nb(n+2) = 1 + \frac{n}{2} = 1 + \frac{n+1}{2} = \frac{n+2}{2}$.

- si n est impair, $n+1$ est pair et $n+2$ est impair. Donc $\frac{n+1}{2} = \frac{n}{2} + 1$ et $\frac{n+2}{2} = \frac{n+1}{2}$.

Par hypothèse de récurrence, $nb(n) = \frac{n}{2}$, donc $nb(n+2) = 1 + \frac{n}{2} = \frac{n+1}{2} = \frac{n+2}{2}$.

On en conclut que $nb(n) = \frac{n}{2}$ pour tout n .

7.2.3 Complexité en moyenne et dans le pire des cas

Soient un algorithme A et une opération fondamentale o , on note $c(\omega)$ le nombre d'opérations o lors de l'exécution de A sur la donnée ω .

On appelle **complexité en temps dans le pire des cas** de A pour o la fonction de $\mathbb{N}$ dans $\mathbb{R}^+$ définie par :

$$C_{pire}(n) \stackrel{\text{def}}{=} \sup\{c(\omega) : |\omega| = n\}$$

On appelle **complexité en temps en moyenne** de A pour o la fonction de $\mathbb{N}$ dans $\mathbb{R}^+$ définie par :

$$C_{moy}(n) \stackrel{\text{def}}{=} \frac{\sum_{|\omega|=n} c(\omega)}{\sum_{|\omega|=n} 1}$$

La complexité dans le pire des cas pour une taille donnée n est donc le nombre d'opérations fondamentales maximales sur une donnée de taille n et la complexité en moyenne pour une taille donnée n est donc le nombre d'opérations fondamentales moyen sur une donnée de taille n .

Chapitre 8

Tri par comparaisons

Contents

8.1	Tri par sélection	47
8.1.1	Présentation	47
8.1.2	Analyse de la complexité en nombre de comparaisons	48
8.1.3	Analyse de la complexité en nombre d'affectations	49
8.2	Tri à bulles	49
8.2.1	Présentation	49
8.2.2	Analyse de la complexité en nombre de comparaisons	50
8.2.3	Analyse de la complexité en nombre d'affectations	51
8.3	Tri par insertion	52
8.3.1	Présentation	52
8.3.2	Analyse de la complexité en nombre de comparaisons	54
8.3.3	Analyse de la complexité en nombre d'affectations	54
8.4	Tri rapide	54
8.4.1	Présentation	54
8.4.2	Analyse de la complexité en nombre de comparaisons	59
8.5	État de l'art	59

Dans ce chapitre et le suivant, nous allons voir de nombreux algorithmes qui résolvent le problème du tri. Ce problème a été beaucoup étudié et il est très important en pratique notamment en informatique de gestion.

Ici, on le réduit en supposant que les tableaux contiennent des entiers mais les mêmes algorithmes s'appliquent aux tableaux dont les éléments sont comparables et que l'on veut classer par ordre croissant. **Cependant, lors des calculs de complexité, il faut garder en mémoire que les éléments du tableau sont des objets complexes dont la manipulation prend plus de temps que celle des entiers.**

Problème du tri

Données : Un tableau de n entiers

Question : Trier le tableau par ordre croissant

Les algorithmes présentés dans ce chapitre pour résoudre le problème du tri sont des algorithmes **par comparaisons**. Ils ont deux opérations fondamentales : la **comparaison entre deux éléments du tableau** et l'**affectation d'une valeur à une case du tableau**. Pour ce problème, la taille de la donnée est n : le nombre d'éléments du tableau.

Le dernier paragraphe de ce chapitre décrit quelle performance on peut attendre pour un tri par comparaisons et indique quels sont les tris par comparaisons les plus efficaces.

8.1 Tri par sélection

8.1.1 Présentation

Le tri par sélection consiste à rechercher le plus petit élément du tableau et à le placer en première position dans le tableau. Puis à rechercher le plus petit des éléments du tableau compris entre la deuxième et la dernière place et à le placer en deuxième position dans le tableau. Et ainsi de suite.

On peut décrire cet algorithme de la façon suivante en supposant que les indices du tableau vont de 0 à $n - 1$:

```
algorithme tri_select
donnee t: tableau de n entiers
variables i,j,k: entiers
debut
  i <- 0;
  tant que i < (n-1) faire
    j <- i;
    pour k variant de (i+1) à n-1 faire
      si (t[k]<t[j])
        alors j<-k;
      fin si
    fin pour
    echanger(t[j],t[i]);
    i <- i+1;
  fin tant que
fin
```

Exemple 26

Les principales étapes de cet algorithme pour le tableau $t=[10, 11, 3, 6, 4, 2]$ sont proposées dans la figure 8.1.

	10	11	3	6	4	2
i=0, j=5	2	11	3	6	4	10
i=1, j=2	2	3	11	6	4	10
i=2, j=4	2	3	4	6	11	10
i=3, j=3	2	3	4	6	11	10
i=4, j=5	2	3	4	6	10	11

FIGURE 8.1 – Tri par sélection de $t=[10, 11, 3, 6, 4, 2]$

8.1.2 Analyse de la complexité en nombre de comparaisons

Pour une taille de tableau donné n , l'algorithme du tri par sélection proposé effectue le même nombre de comparaisons quelle que soit la valeur des éléments contenus dans le tableau. Par conséquent, la complexité en moyenne de cet algorithme est la même que celle dans le pire des cas pour le nombre de comparaisons.

L'algorithme du tri par sélection proposé effectue une comparaison entre deux éléments du tableau pour chaque valeur de k dans la boucle **pour**. Cela représente $n-1 - (i+1) + 1 = n-(i+1)$ comparaisons pour l'ensemble de la boucle **pour**.

Il y a un appel à la boucle **pour** pour chaque valeur de i , on en conclut donc que

$$C_{moy}(n) = C_{pire}(n) = \sum_{i=0}^{n-2} n - (i + 1)$$

On fait un changement de variable : $j = n - (i + 1)$

$$C_{moy}(n) = C_{pire}(n) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2)$$

Rappelons que $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

8.1.3 Analyse de la complexité en nombre d'affectations

Les affectations d'une valeur à une case du tableau sont effectuées lors de l'appel à `échange`. On peut écrire une fonction `échange` qui comprend trois affectations d'une valeur d'une case du tableau.

Pour une taille de tableau donné n , l'algorithme du tri par sélection proposé effectue le même nombre d'échanges et donc d'affectations quelle que soit la valeur des éléments contenus dans le tableau. Par conséquent, la complexité en moyenne de cet algorithme est la même que celle dans le pire des cas pour le nombre d'affectations.

L'algorithme du tri par sélection proposé effectue un échange par valeur de i . Donc :

$$C_{moy}(n) = C_{pire}(n) = 3 \times (n - 1) = \Theta(n)$$

8.2 Tri à bulles

8.2.1 Présentation

L'idée du tri à bulles est que les éléments les plus petits (les plus légers) remontent au début du tableau. Concrètement, on parcourt le tableau en commençant par la fin et en effectuant un échange à chaque fois qu'on trouve deux éléments consécutifs qui ne sont pas dans le bon ordre. Après le premier passage, l'élément le plus petit se retrouve dans la première case. On parcourt à nouveau le tableau mais en s'arrêtant à la deuxième case et ainsi de suite.

On peut décrire cet algorithme de la façon suivante :

```
algorithme tri_bulles
donnee t: tableau de n entiers
variables i,j: entiers
debut
  i <- 0;
  tant que i < (n-1) faire
    pour j variant de n-1 à (i+1) faire
      si (t[j]<t[j-1])
        echanger(t[j-1],t[j]);
      fin si
    fin pour
    i <- i+1;
  fin tant que
```

fin

Exemple 27

Les principales étapes de cet algorithme pour le tableau $t=[10, 11, 3, 6, 4, 2]$ sont proposées dans la figure 8.2.

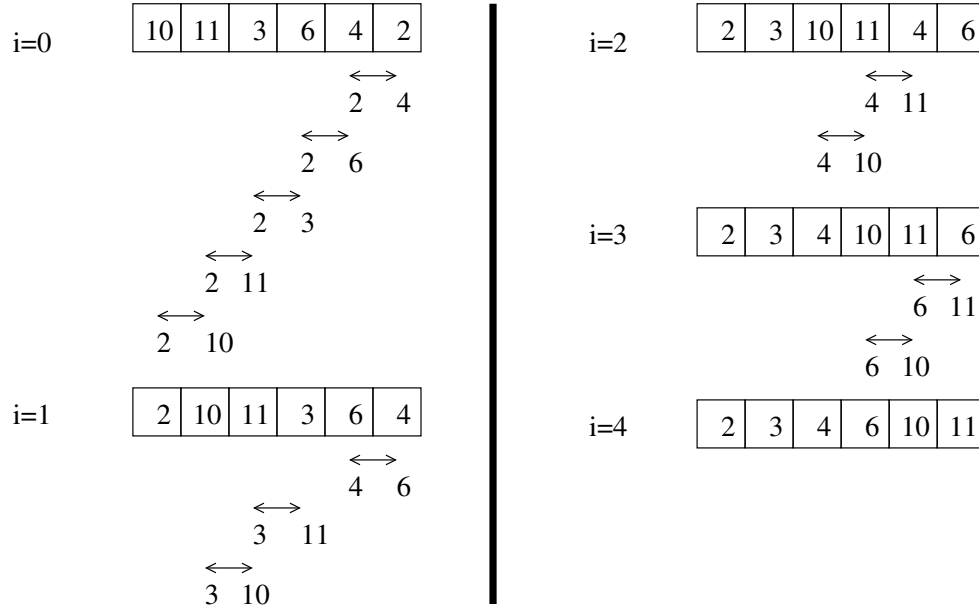


FIGURE 8.2 – Tri à bulles de $t=[10, 11, 3, 6, 4, 2]$.

8.2.2 Analyse de la complexité en nombre de comparaisons

Pour une taille de tableau donné n , l'algorithme du tri à bulles proposé effectue le même nombre de comparaisons quelle que soit la valeur des éléments contenus dans le tableau. Par conséquent, la complexité en moyenne de cet algorithme est la même que celle dans le pire des cas pour le nombre de comparaisons. De plus, le calcul est quasi-identique à celui du tri par sélection.

L'algorithme du tri à bulles proposé effectue une comparaison entre deux éléments du tableau pour chaque valeur de j dans la boucle **pour**. Cela représente $n-1 - (i+1) + 1 = n-(i+1)$ comparaisons pour l'ensemble de la boucle **pour**.

Il y a un appel à la boucle **pour** pour chaque valeur de i , on en conclut donc que

$$C_{moy}(n) = C_{pire}(n) = \sum_{i=0}^{n-2} n - (i + 1)$$

On fait un changement de variable : $k = n - (i + 1)$.

$$C_{moy}(n) = C_{pire}(n) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2)$$

8.2.3 Analyse de la complexité en nombre d'affectations

Comme dans le cas du tri par sélection, les affectations d'une valeur à une case du tableau sont effectuées lors de l'appel à **échange**.

Contrairement aux cas précédents, pour une taille de tableau donné n , le nombre d'échanges lors de l'exécution de l'algorithme du tri à bulles proposé dépend des éléments contenus dans le tableau. Nous allons commencer par déterminer la complexité de cet algorithme dans le pire des cas pour le nombre d'affectations avant d'étudier la complexité en moyenne.

8.2.3.1 Complexité en nombre d'affectations dans le pire des cas

Dans le pire des cas, il y a un échange à chaque fois qu'il y a une comparaison. Ce cas est obtenu quand le tableau donné est ordonné par ordre décroissant. On a ainsi

$$C_{pire}(n) = \Theta(n^2)$$

8.2.3.2 Complexité en nombre d'affectations en moyenne

Le calcul de la complexité en moyenne est un peu plus compliqué que les calculs précédents. Nous supposons que tous les éléments du tableau sont distincts.

Lors de l'exécution du tri à bulles, si $i < j$ et $a = t[i] > t[j] = b$, alors a et b sont échangés exactement une fois : au moins une fois car le tableau n'est pas trié sinon ; et, au plus une fois car deux éléments dans le bon ordre ne sont jamais échangés. On note $nb_E(t)$ le nombre d'échanges lors de l'exécution du tri à bulles sur t .

Soit t un tableau de taille n , on définit son **miroir** t' de la façon suivante : $t'[i] = t[n - 1 - i]$.

Exemple 28

Si $t = [10, 11, 3, 6, 4, 2]$, $t' = [2, 4, 6, 3, 11, 10]$.

Si on exécute l'algorithme de tri à bulles sur t et t' , chaque paire d'éléments est échangée soit dans t , soit dans t' , mais jamais dans les deux. Lors de l'exécution du tri à bulles sur les tableaux t et t' , on a en tout autant d'échanges qu'il y a de paires d'éléments d'indices différents, soit $\frac{n(n-1)}{2}$ échanges.

Soit T l'ensemble de tous les tableaux de taille n ne comportant pas d'éléments égaux. On suppose qu'ils sont tous équiprobables de probabilité p , alors :

$$C_{moy}(n) = \sum_{t \in T} p \times 3 \times nb_E(t) = 3 \times p \times \sum_{t \in T} nb_E(t)$$

On partitionne T en T_c et T_d :

$$\begin{aligned} T_c &= \{t \in T \mid t[1] < t[n]\} \\ T_d &= \{t \in T \mid t[1] > t[n]\} \end{aligned}$$

On a $t \in T_c$ si et seulement si $t' \in T_d$. Il s'ensuit que :

$$\begin{aligned} C_{moy}(n) &= 3 \times p \times \sum_{t \in T_c} nb_E(t) + 3 \times p \times \sum_{t \in T_d} nb_E(t) \\ &= 3 \times p \times \sum_{t \in T_c} nb_E(t) + 3 \times p \times \sum_{t \in T_c} nb_E(t') \\ &= 3 \times p \times \sum_{t \in T_c} (nb_E(t) + nb_E(t')) \\ &= 3 \times p \times \sum_{t \in T_c} \frac{n(n-1)}{2} \\ &= 3 \times p \times |T_c| \times \frac{n(n-1)}{2} \end{aligned}$$

$|T_c|$ désigne le nombre d'éléments de T_c , comme $t \in T_c$ si et seulement si $t' \in T_d$, $|T_c| = |T|/2$.

De plus, comme tous les tableaux sont équiprobables, $p = 1/|T|$. Donc :

$$C_{moy}(n) = \frac{3 \times n(n-1)}{4} = \Theta(n^2)$$

8.3 Tri par insertion

8.3.1 Présentation

Ce tri correspond à la manière de faire de la plupart des joueurs de cartes : on insère les cartes au fur à mesure à leur place en décalant celles qui sont déjà ordonnées.

On peut décrire cet algorithme de la façon suivante :

```

algorithme tri_insertion
donnee t: tableau de n entiers
variables k,x,i: entiers
          b: booleen
debut
  pour i variant de 1 à n-1 faire
    k <- i-1;
    x <- t[i];
    b <- (t[k]>x);
    tant que (b) faire
      t[k+1] <- t[k];
      k <- k-1;
    si (k > -1)
      alors b <- (t[k]>x);
      sinon b <- faux;
    fin si
  fin tant que
  t[k+1] <- x;
fin

```

Exemple 29

Les principales étapes de cet algorithme pour le tableau $t=[10, 11, 3, 6, 4, 2]$ sont proposées dans la figure 8.3.

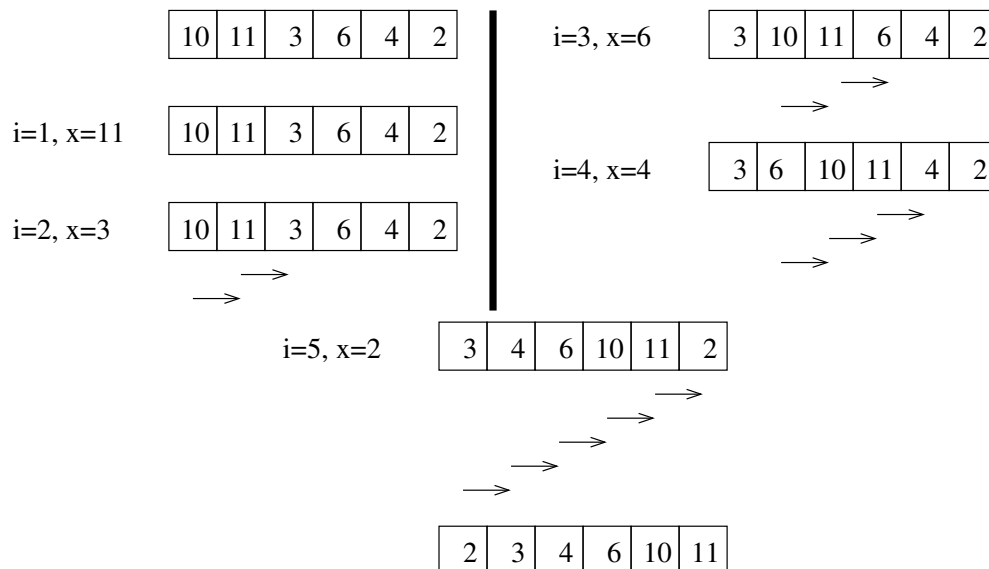


FIGURE 8.3 – Tri par insertion de $t=[10, 11, 3, 6, 4, 2]$.

8.3.2 Analyse de la complexité en nombre de comparaisons

Contrairement aux cas précédents, pour une taille de tableau donné n , le nombre de comparaisons lors de l'exécution de l'algorithme du tri par insertion proposé dépend des éléments contenus dans le tableau. Nous allons commencer par déterminer la complexité de cet algorithme dans le pire des cas pour le nombre de comparaisons avant d'étudier la complexité en moyenne.

La complexité dans le pire des cas est obtenue à nouveau pour un tableau ordonné par ordre décroissant et on a alors :

$$C_{pire}(n) = \Theta(n^2)$$

Pour la complexité en moyenne, on l'obtient en faisant un raisonnement proche de celui pour la complexité en moyenne pour le nombre d'affectations pour le tri à bulles. D'où

$$C_{moy}(n) = \Theta(n^2)$$

8.3.3 Analyse de la complexité en nombre d'affectations

Pour chaque valeur de i , le nombre d'affectations diffère au plus de 1 par rapport au nombre de comparaisons d'où

$$C_{pire}(n) = \Theta(n^2)$$

$$C_{moy}(n) = \Theta(n^2)$$

8.4 Tri rapide

L'idée consiste à regarder le premier élément du tableau, à déplacer à sa gauche les éléments plus petits que lui et à sa droite les éléments plus grands, puis à recommencer avec le tableau formé par les éléments de gauche d'une part, et le tableau formé par les éléments de droite d'autre part.

8.4.1 Présentation

```
algorithme tri_rapide
donnee t: tableau de n entiers
debut
    rapide(t,0,n-1);
fin
```



```

procedure rapide(tab: tableau de n entiers, i: entier, j: entier)
variable aux: entier
debut
  si (i < j)
    alors
      aux <- partitionner(tab,i,j);
      rapide(tab,i,aux-1);
      rapide(tab,aux+1,j);
    fin si
fin

fonction partitionner(tab: tableau de n entiers, i: entier, j: entier):
entier
variables x,l,m: entiers;
          b,bb: booleen
debut
  x <- tab[i];
  l <- i+1;
  m <- j;
  b <- vrai;
  tant que (b) faire
    tant que (tab[m] > x) faire
      m <- m-1;
    fin tant que
    bb <- (tab[l] <= x);
    tant que (bb) faire
      l <- l+1;
      si (l <= j)
        alors
          bb <- (tab[l] <= x);
        sinon
          bb <- faux;
        fin si
    fin tant que
    si (l < m)
      alors
        echanger(tab[l], tab[m]);
      sinon
        b <- faux;
      fin si
  fin si

```

```

    fin tant que
    echanger(tab[i],tab[m]);
    retourner m;
fin

```

On regarde le premier élément du tableau avec l'instruction $x \leftarrow \text{tab}[i]$; de **partitionner**.

Puis toute l'exécution de **partitionner** consiste à déplacer à la gauche de x les éléments plus petits que lui et à sa droite les éléments plus grands.

On recommence récursivement ces opérations d'une part avec le tableau formé par les éléments désormais à gauche de x avec l'appel **rapide**(tab,i,aux-1); et d'autre part avec le tableau formé par les éléments à droite de x avec l'appel **rapide**(tab,aux+1,j); dans **rapide**.

Exemple 30

Nous allons voir les principales étapes de cet algorithme pour le tableau $t=[10, 11, 3, 6, 4, 2]$.

Tout d'abord, l'appel de **rapide**($t, 0, 5$) lance l'exécution de **partitionner**($t, 0, 5$). On procède aux initialisations de x , l , m et b et on est dans la situation de la figure 8.4.

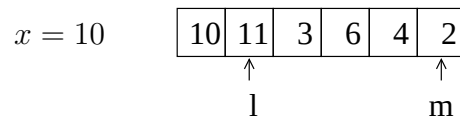


FIGURE 8.4 – Après l'initialisation de **partitionner**($t, 0, 5$).

Les deux boucles **tant que** qui suivent ne modifient pas les valeurs de m et l . Par conséquent, comme ($l < m$), on échange $\text{tab}[l]$ et $\text{tab}[m]$. On est dans la situation de la figure 8.5.

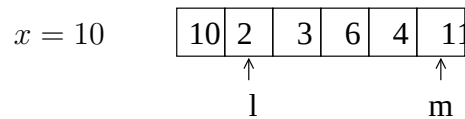


FIGURE 8.5 – Après l'échange de $\text{tab}[l]$ et $\text{tab}[m]$.

b est toujours vraie, donc on effectue la boucle **tant que** portant sur $\text{tab}[m]$ qui décrémente m jusqu'à 4 et la boucle **tant que** portant sur $\text{tab}[l]$ qui incrémente l jusqu'à 5. On est dans la situation de la figure 8.6.

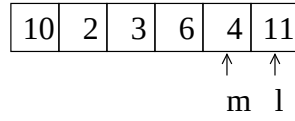


FIGURE 8.6 – Après le deuxième passage dans la boucle portant sur b .

Comme ($l \geq m$), on modifie la valeur de b . Par conséquent, on sort de la boucle **tant que** portant sur b . On échange $tab[i]$ et $tab[m]$ et on sort de **partitionner**($t, 0, 5$) en retournant la valeur 4. On est dans la situation de la figure 8.7.

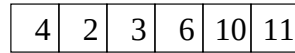


FIGURE 8.7 – Après **partitionner**($t, 0, 5$).

Comme $aux=4$, on doit maintenant exécuter **rapide**($t, 0, 3$), puis **rapide**($t, 5, 5$). On peut remarquer que **rapide**($t, 5, 5$) n'exécute aucune instruction.

L'appel de **rapide**($t, 0, 3$) lance l'exécution de **partitionner**($t, 0, 3$). On procède aux initialisations de x , l , m et b et on est dans la situation de la figure 8.8.

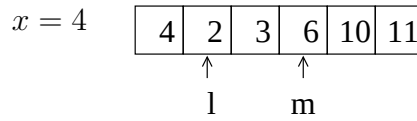


FIGURE 8.8 – Après l'initialisation de **partitionner**($t, 0, 3$).

On effectue la boucle **tant que** portant sur $tab[m]$ qui décrémente m jusqu'à 2 et la boucle **tant que** portant sur $tab[l]$ qui incrémente l jusqu'à 3. On est dans la situation de la figure 8.9.

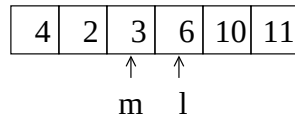


FIGURE 8.9 – Après le premier passage dans la boucle portant sur b .

Comme ($l \geq m$), on modifie la valeur de b . Par conséquent, on sort de la boucle **tant que** portant sur b . On échange $tab[i]$ et $tab[m]$, puis on sort de **partitionner**($t, 0, 3$) en retournant la valeur 2. On est dans la situation de la figure 8.10.

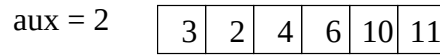


FIGURE 8.10 – Après `partitionner(t, 0, 3)`.

aux=2, donc on doit maintenant exécuter `rapide(t, 0, 1)` et puis `rapide(t, 3, 3)`. On peut remarquer que `rapide(t, 3, 3)` n'exécute aucune instruction.

L'appel de `rapide(t, 0, 1)` lance l'exécution de `partitionner(t, 0, 1)`. On procède aux initialisations de x , l , m et b et on est dans la situation de la figure 8.11.

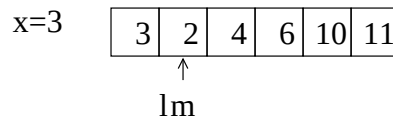


FIGURE 8.11 – Après l'initialisation de `partitionner(t, 0, 1)`.

On effectue la boucle **tant que** portant sur `tab[m]` qui ne modifie pas m et la boucle **tant que** portant sur `tab[l]` qui incrémente l jusqu'à 2. On est dans la situation de la figure 8.12.

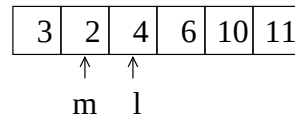


FIGURE 8.12 – Après le premier passage dans la boucle portant sur b .

Comme ($l \geq m$), on modifie la valeur de b . Par conséquent, on sort de la boucle **tant que** portant sur b . On échange `tab[i]` et `tab[m]`, puis on sort de `partitionner(t, 0, 1)` en retournant la valeur 1. On est dans la situation de la figure 8.13.

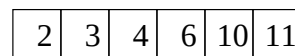


FIGURE 8.13 – Après `partitionner(t, 0, 1)`.

aux=1, donc on doit maintenant exécuter `rapide(t, 0, 0)` et puis `rapide(t, 2, 1)`. On peut remarquer que `rapide(t, 0, 0)` et `rapide(t, 2, 1)` n'exécutent aucune instruction. On a ainsi fini.

8.4.2 Analyse de la complexité en nombre de comparaisons

Pour une taille de tableau donné n , le nombre de comparaisons lors de l'exécution de l'algorithme du tri rapide proposé dépend des éléments contenus dans le tableau.

La complexité dans le pire des cas est obtenue à nouveau pour un tableau ordonné. On a alors :

$$C_{pire}(n) = \Theta(n^2)$$

La complexité en moyenne est

$$C_{moyenne}(n) = \Theta(n \log n)$$

Le fait que la complexité dans le pire des cas soit obtenu pour les tableaux triés est un vrai problème car en pratique, on a souvent à trier des tableaux presque ordonnés. On remédie à ce problème en ne choisissant plus comme pivot (x) le premier élément du tableau mais l'élément médian entre le premier élément du tableau, le dernier et celui du milieu.

8.5 État de l'art

Il existe des tris par comparaisons dont la complexité en moyenne comme dans le pire des cas en nombre de comparaisons est en $\Theta(n \log n)$. C'est le cas en particulier du tri par tas qui est présenté dans le chapitre sur les arbres binaires.

Il a été démontré qu'un algorithme doit faire au moins $\Theta(n \log n)$ comparaisons pour trier un tableau. Par conséquent, le tri par tas est optimal.

En pratique, on utilise le tri rapide avec comme pivot l'élément médian entre le premier élément du tableau, le dernier et celui du milieu. Les résultats sont alors souvent meilleurs que ceux du tri par tas mais ils sont en $\Theta(n^2)$ dans le pire des cas.

Chapitre 9

Tri par dénombrement

Contents

9.1	Tri par dénombrement	60
9.1.1	Présentation	60
9.1.2	Analyse de la complexité	65
9.2	Application : tri par base	65

Dans ce chapitre, nous allons voir des tris de complexité linéaire (c'est-à-dire en $\Theta(n)$). Cela peut paraître contradictoire avec le fait que les tris par comparaison ont une complexité qui ne peut pas être meilleure que $\Theta(n \log n)$ en nombre de comparaisons. En fait, les tris de ce chapitre n'ont pas comme opération fondamentale la comparaison entre deux éléments du tableau. En contrepartie de cette meilleure performance, les hypothèses d'application sont plus contraignantes.

9.1 Tri par dénombrement

9.1.1 Présentation

Le tri par dénombrement ne s'applique que dans le cas où les n entiers du tableau à trier sont compris entre 1 et k où $k = O(n)$. Le principe du tri par dénombrement est de déterminer, pour chaque élément x du tableau, le nombre d'éléments inférieurs à x . Cette information permet de déterminer quelle est la place de x dans le tableau trié. Le résultat du tri apparaît dans le tableau s .

```
algorithme tri_dénombrement
donnee t,s: tableau de n entiers
      k: entier
variables c: tableau de k+1 entiers
```

```

        i,j: entiers
debut
  pour i variant de 0 à k faire
    c[i] <- 0;
  fin pour
  pour j variant de 0 à n-1 faire
    c[t[j]] <- c[t[j]] + 1
  fin pour
  pour i variant de 1 à k faire
    c[i] <- c[i]+c[i-1];
  fin pour
  pour j variant de n-1 à 0 faire
    s[c[t[j]]-1] <- t[j]
    c[t[j]] <- c[t[j]] - 1
  fin pour

```

Exemple 31

Nous allons voir les principales étapes de cet algorithme pour le tableau $t = [3, 2, 4, 4, 5, 6, 4, 2]$:

1. La première boucle de l'algorithme remplit le tableau c de 0.
2. La deuxième boucle de l'algorithme remplit le tableau c de façon à ce que $c[i]$ soit égal au nombre de fois où i apparaît dans le tableau t . La figure 9.1 illustre les modifications de c lors de l'exécution de la deuxième boucle de l'algorithme.

$c[t[0]] \leftarrow -c[t[0]] + 1 :$	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	1	0	0	0
0	0	0	1	0	0	0		
$c[t[1]] \leftarrow -c[t[1]] + 1 :$	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	1	1	0	0	0
0	0	1	1	0	0	0		
$c[t[2]] \leftarrow -c[t[2]] + 1 :$	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>	0	0	1	1	1	0	0
0	0	1	1	1	0	0		
$c[t[3]] \leftarrow -c[t[3]] + 1 :$	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>0</td><td>0</td></tr></table>	0	0	1	1	2	0	0
0	0	1	1	2	0	0		
$c[t[4]] \leftarrow -c[t[4]] + 1 :$	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>1</td><td>0</td></tr></table>	0	0	1	1	2	1	0
0	0	1	1	2	1	0		
$c[t[5]] \leftarrow -c[t[5]] + 1 :$	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>1</td><td>1</td></tr></table>	0	0	1	1	2	1	1
0	0	1	1	2	1	1		
$c[t[6]] \leftarrow -c[t[6]] + 1 :$	<table><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>3</td><td>1</td><td>1</td></tr></table>	0	0	1	1	3	1	1
0	0	1	1	3	1	1		
$c[t[7]] \leftarrow -c[t[7]] + 1 :$	<table><tr><td>0</td><td>0</td><td>2</td><td>1</td><td>3</td><td>1</td><td>1</td></tr></table>	0	0	2	1	3	1	1
0	0	2	1	3	1	1		

FIGURE 9.1 – Exécution de la deuxième boucle.

3. La troisième boucle de l'algorithme modifie le tableau c de façon à ce que $c[i]$ soit égal au nombre de fois où un élément inférieur ou égal à i apparaît dans le tableau t . La figure 9.2 illustre les modifications de c lors de l'exécution de la troisième boucle de l'algorithme.

	0	0	2	1	3	1	1
$c[1] \leftarrow -c[1] + c[0] :$	0	0	2	1	3	1	1
$c[2] \leftarrow -c[2] + c[1] :$	0	0	2	1	3	1	1
$c[3] \leftarrow -c[3] + c[2] :$	0	0	2	3	3	1	1
$c[4] \leftarrow -c[4] + c[3] :$	0	0	2	3	6	1	1
$c[5] \leftarrow -c[5] + c[4] :$	0	0	2	3	6	7	1
$c[6] \leftarrow -c[6] + c[5] :$	0	0	2	3	6	7	8

FIGURE 9.2 – Exécution de la troisième boucle.

À la fin de son exécution, $c = [0, 0, 2, 3, 6, 7, 8]$.

4. La dernière boucle permet de remplir le tableau s qui contient au final les éléments triés par ordre croissant. Les figures 9.3 et 9.4 illustrent les modifications de s et c lors de l'exécution de la quatrième boucle de l'algorithme.

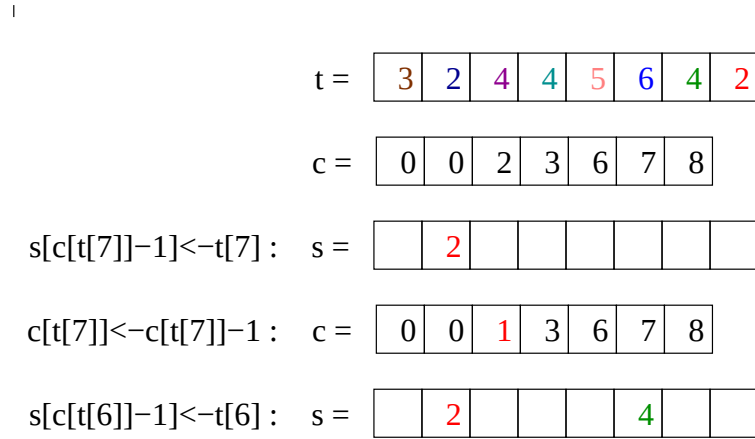


FIGURE 9.3 – Exécution de la quatrième boucle (début).

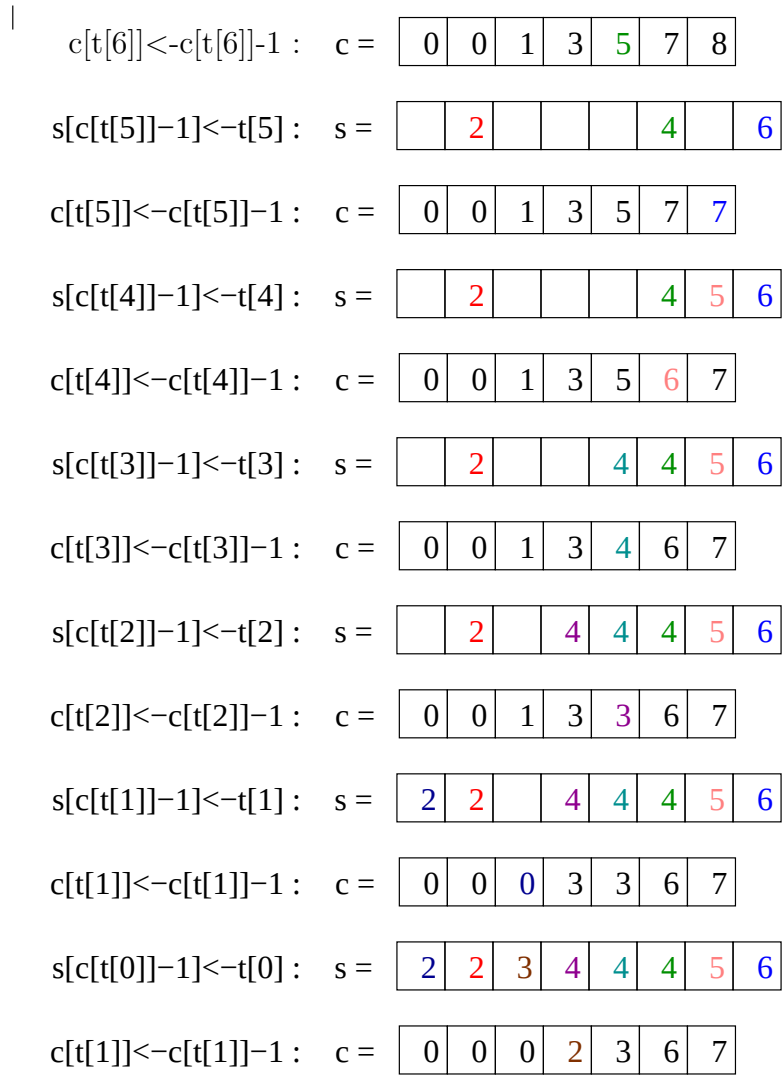


FIGURE 9.4 – Exécution de la quatrième boucle (fin).

Sur ces figures apparaît une propriété importante la **stabilité** : les occurrences d'un même entier apparaissent dans le même ordre dans s et dans t .

Exemple 32

On voit dans la figure 9.4 que les 4 et les 2 ont des couleurs ordonnées de la même façon dans s et t .

Le tri par dénombrement n'est pas un tri sur place : il y a deux tableaux, un tableau donné t et un tableau résultat s .

9.1.2 Analyse de la complexité

On considère comme opération fondamentale l'affectation d'une valeur à une case d'un tableau.

Pour une taille de tableau donnée n , l'algorithme de tri par dénombrement proposé effectue le même nombre d'affectations quelle que soit la valeur des éléments contenus dans le tableau. Par conséquent, la complexité en moyenne de cet algorithme est la même que celle dans le pire des cas pour le nombre de d'affectations.

1. Il y a une affectation pour chaque valeur de i dans la première boucle, soit $k + 1$ affectations.
2. Il y a une affectation pour chaque valeur de j dans la première boucle, soit n affectations.
3. Il y a une affectation pour chaque valeur de i dans la troisième boucle, soit k affectations.
4. Il y a deux affectations pour chaque valeur de j dans la quatrième boucle, soit $2n$ affectations.

Donc, $C_{moy}(n) = C_{pire}(n) = 3n + 2k + 1$. Comme $k = \Theta(n)$,

$$C_{moy}(n) = C_{pire}(n) = \Theta(n)$$

9.2 Application : tri par base

On suppose que l'on a un tableau de n entiers écrits en base k avec $k = \Theta(n)$ et que chaque entier s'écrit avec au plus m symboles. Le principe du tri par base consiste à trier m fois le tableau à l'aide du tri par dénombrement en commençant par trier les entiers par rapport à leur chiffre des unités, puis celui des dizaines, etc.

Exemple 33

Les figures 9.5 et 9.6 montre les tris successifs du tri par base.

	4567	3578	2468	1254	23	6523	4784	6426	786
tri des unités:	23	6523	1254	4784	6426	786	4567	3578	2468
tri des dizaines:	23	6523	6426	1254	4567	2468	3578	4784	786

FIGURE 9.5 – Tri par base (début).

tri des centaines:	23	1254	6426	2468	6523	4567	3578	4784	786
tri des milliers:	23	786	1254	2468	3578	4567	4784	6426	6523

FIGURE 9.6 – Tri par base (*fin*).

La stabilité du tri par dénombrement permet de conserver les résultats des tris précédents lors d'un nouveau tri. Sans cette propriété, le tri selon les dizaines ne profiterait pas du tri sur les unités qui serait alors perdu.

Exemple 34

Soit le tableau $t = [41, 44, 36]$ déjà trié sur les unités. Un tri non-stable sur les dizaines pourrait retourner $t = [36, 44, 41]$.

On considère comme opération fondamentale l'affectation d'une valeur à une case d'un tableau. L'algorithme de tri par base effectue le même nombre d'affectations quelle que soit la valeur des éléments contenus dans le tableau.

Donc, $C_{moy}(n) = C_{pire}(n) = \Theta(mn)$.

Chapitre 10

Problèmes $\mathcal{NP}$ -complets

Contents

10.1 Introduction	68
10.2 Le Sudoku est un problème $\mathcal{NP}$	69
10.3 Présentation	69
10.3.1 Un algorithme $\mathcal{NP}$ pour le problème du Sudoku	70
10.3.2 Un algorithme déterministe	71
10.4 Transformations polynomiales	71
10.4.1 Définition	71
10.4.2 Propriétés	72
10.5 Problèmes $\mathcal{NP}$-complets	72
10.6 SAT	73
10.6.1 Calcul booléen	73
10.6.2 Calcul propositionnel	74
10.6.3 Théorème de Cook	74

On a vu au chapitre 6 (Classes de complexité) les classes $\mathcal{P}$ et $\mathcal{NP}$. L'objectif de ce chapitre est d'étudier plus en détail ces deux classes.

Pour les questions théoriques, un problème appartient à la classe $\mathcal{P}$ s'il existe une machine de Turing déterministe M qui le décide telle que $C_t(n) = O(f(n))$ où f est une fonction polynomiale.

Dans les cas pratiques, un problème appartient à la classe $\mathcal{P}$ s'il existe un algorithme qui le décide dont la complexité en temps dans le pire des cas est polynomiale pour l'opérations fondamentale.

Ces deux définitions sont équivalentes d'après la thèse de l'invariance (section 6.3) à condition d'avoir correctement choisi l'opération fondamentale.

10.1 Introduction

Il y a de cela quelques années, la Fondation Clay¹ a promis la somme d'un million de dollars à quiconque résoudrait l'une des sept conjectures mathématiques majeures, c'est-à-dire ayant des implications scientifiques très larges. Depuis, seule une, la conjecture de Poincaré a été résolue².

L'une de ces conjectures est $\mathcal{P}$ vs $\mathcal{NP}$ ³. Autrement dit, **est-ce que les classes $\mathcal{P}$ et $\mathcal{NP}$ sont égales ?**

Intuitivement, un problème $\mathcal{P}$ est un problème que l'on peut résoudre par une méthode qui demande peu d'opérations (un nombre polynomial par rapport à la taille des données). Un problème $\mathcal{NP}$ -dur (ce terme sera précisé plus tard) est un problème qui demande beaucoup d'opérations mais dont la vérification de la solution demande peu d'opérations. Cette notion intuitive de problème facile à calculer ou facile à vérifier a pu être formalisée grâce aux travaux de Turing.

Quand on a un problème dont on ne connaît qu'une solution nécessitant beaucoup d'opérations, on peut se demander s'il n'existe pas une autre solution plus simple. Et plus largement, **existe-t-il des problèmes dont toutes les solutions nécessitent beaucoup d'opérations ?** Intuitivement, la majorité des personnes pensent que oui mais on ne sait pas le montrer.

Savoir si un problème est difficile a deux applications majeures :

- Lorsque l'on souhaite garantir la sécurité d'une application, par exemple qu'un message codé ne peut pas être déchiffré en un temps raisonnable, il convient de prouver que le problème du codage est très difficile.
- Lorsqu'un problème est identifié comme difficile, chercher un algorithme efficace est utopique.

Par ailleurs, un algorithme naïf sera en général inutilisable. Par exemple, le problème de l'emploi du temps (on se donne n classes, p professeurs, m salles, avec des contraintes sur les volumes horaires et il faut trouver un emploi du temps compatible) est difficile. Si l'on fait un programme qui explore toutes les possibilités, on obtient un programme qui ne trouvera rapidement des solutions que si les contraintes sont très faibles, par exemple beaucoup de salles pour peu de classes et de professeurs. Mais dans les cas plus conséquents, il pourra mettre des années à trouver une solution.

Il convient dans ce cas, non pas d'essayer de résoudre le problème, mais de recoder son problème en un autre problème équivalent pour lequel il

1. <http://www.claymath.org/millennium-problems>

2. http://www.lefigaro.fr/sciences/2006/12/23/01008-20061223ARTWWW90012-decouverte_la_conjecture_de_poincare.php

3. <http://www.claymath.org/millennium-problems/p-vs-np-problem>

existe un programme optimisé et contenant des dizaines d'heuristiques qui fonctionnent en pratique. En général, ce genre de programme prend en entrée des formules logiques et sont appelés des solveurs.

À partir de la définition des deux classes $\mathcal{P}$ et $\mathcal{NP}$, on a cherché à classer finement les problèmes en classe de complexité. On trouve sur https://complexityzoo.uwaterloo.ca/Complexity_Zoo la description de plus de 460 classes différentes.

10.2 Le Sudoku est un problème $\mathcal{NP}$

10.3 Présentation

Problème : Sudoku

Données : un entier n qui est un carré, sa racine carré y , une grille de n cases sur n , divisée en n groupes de y cases sur y et partiellement remplie de nombres compris entre 1 et n .

Question : Peut-on achever de remplir la grille avec des nombres compris entre 1 et n tel qu'un même nombre n'apparaisse jamais deux fois dans une même ligne, une même colonne ou un même groupe ?

Exemple 35

On considère l'instance suivante : $n=9$, $y=3$ et la grille est présentée par la figure 10.1.

			1		2			
		9	4			2		6
7	2	6		9	5	4		8
2	8		9	5				3
						6		
4		3	7		8	5		2
6	3			2	9	1		
		2		4	3	8	6	
		4			1		2	

FIGURE 10.1 – Donnée.

La réponse est oui, la figure 10.2 propose une façon de compléter la grille satisfaisant les critères.

3	4	8	1	6	2	9	5	7
5	1	9	4	8	7	2	3	6
7	2	6	3	9	5	4	1	8
2	8	1	9	5	6	7	4	3
9	5	7	2	3	4	6	8	1
4	6	3	7	1	8	5	9	2
6	3	5	8	2	9	1	7	4
1	7	2	5	4	3	8	6	9
8	9	4	6	7	1	3	2	5

FIGURE 10.2 – Solution.

Dans la figure 10.3, le chiffre 3 est indiqué dans cette grille terminée, pour vous montrer qu'il respecte bien les règles de base du Sudoku. Il figure bien 9 fois et une seule fois seulement dans chaque unité. C'est aussi le cas pour les autres chiffres.

3	4	8	1	6	2	9	5	7
5	1	9	4	8	7	2	3	6
7	2	6	3	9	5	4	1	8
2	8	1	9	5	6	7	4	3
9	5	7	2	3	4	6	8	1
4	6	3	7	1	8	5	9	2
6	3	5	8	2	9	1	7	4
1	7	2	5	4	3	8	6	9
8	9	4	6	7	1	3	2	5

FIGURE 10.3 – La situation du 3

10.3.1 Un algorithme $\mathcal{NP}$ pour le problème du Sudoku

Un algorithme qui résout le problème du Sudoku consiste à choisir une valeur comprise entre 1 et n pour chacune des cases vides de la grille donnée puis de vérifier que la grille ainsi obtenue convient. Si c'est le cas, cette exécution répond oui au problème, sinon non.

Cet algorithme est non-déterministe puisqu'on a le choix entre n chemins à chaque fois qu'on choisit la valeur d'une case.

Il résout bien le problème puisque l'on envisage toutes les façons de remplir la grille. Donc s'il y a une solution vérifiant tous les critères, elle correspond à l'un des chemins et l'algorithme répondra oui. S'il n'y a pas de solution, tous les chemins répondront non.

Si on choisit comme opération fondamentale la comparaison d'entiers, cet algorithme est polynomial. On peut vérifier qu'une ligne ne contient pas deux fois le même nombre en faisant une double boucle dessus.

```

pour i variant de 1 à n
  pour j variant de 1 à n
    si i != j alors
      si la ième valeur de la ligne = la jème valeur de la ligne alors
        cette grille ne convient pas
      fin si
    fin si
  fin pour
fin pour

```

On fait alors au plus 2 comparaisons par paire (i, j) ce qui est un $O(n^2)$.

Il y a n lignes donc $O(n^3)$ comparaisons pour toutes les lignes. Idem pour les colonnes et les carrés. Ce qui fait un total de $O(n^3)$ comparaisons pour la vérification et donc pour l'algorithme.

On en conclut que le problème du Sudoku est un problème de la classe $\mathcal{NP}$.

10.3.2 Un algorithme déterministe

Comme on l'a vu au chapitre 4 (Variations sur la machine de Turing), on peut transformer l'algorithme précédent en un algorithme déterministe. Dans le cas présent, on va explorer toutes les façons de remplir la grille jusqu'à ce qu'on en trouve une pour laquelle la vérification retourne oui.

Il y a au plus n^2 cases à remplir avec n choix pour chacune, ceci fait au plus $n^{(n^2)}$ grilles différentes. Pour chacun, il y a $O(n^3)$ comparaisons pour la vérification. Donc cet algorithme déterministe est en $O(n^{(n^2+3)})$. Tel quel, il est inutilisable.

10.4 Transformations polynomiales

10.4.1 Définition

Soient un langage $L_1 \in \Sigma_1^*$ et un langage $L_2 \in \Sigma_2^*$. Une **transformation polynomiale** de L_1 vers L_2 est une fonction $f : \Sigma_1^* \rightarrow \Sigma_2^*$ qui satisfait les conditions

suivantes :

- elle est décidable en temps polynomial
- $x \in L_1$ si et seulement si $f(x) \in L_2$

Quand il existe une transformation polynomiale de L_1 vers L_2 , on note $L_1 \alpha L_2$ cette propriété. On nomme aussi réductions polynomiales les transformations polynomiales.

10.4.2 Propriétés

Si $L_1 \alpha L_2$, alors

- si $L_2 \in \mathcal{P}$, alors $L_1 \in \mathcal{P}$
- si $L_1 \notin \mathcal{P}$, alors $L_2 \notin \mathcal{P}$

Le second point est la contraposée du premier (*modus tollens*), il suffit donc de montrer que si $L_2 \in \mathcal{P}$, alors $L_1 \in \mathcal{P}$.

Supposons qu'il existe une machine de Turing déterministe polynomiale M qui décide le langage L_2 et soit f la réduction polynomiale $L_1 \alpha L_2$. Pour décider si un mot x appartient à L_1 , il suffit de calculer $f(x)$ en temps polynomial, de le mettre en entrée de M et de faire le calcul.

Nous allons maintenant montrer que ce calcul est polynomial. Supposons que f ait une complexité $c(n)$ et M une complexité $c'(n)$. Pour un mot de longueur n , la réduction polynomiale aura donc un temps de calcul $c(n)$ et produira un résultat de longueur au plus $c(n)$ (le résultat est de longueur inférieure au temps de calcul puisque l'écriture de chaque symbole du résultat nécessite au moins une étape de calcul). Le temps de calcul de M sur $f(n)$ sera donc inférieur ou égal à $c'(c(n))$. Donc on aura décidé le langage L_2 en un temps inférieur à $c(n) + c'(c(n))$ qui est un polynôme en n .

10.5 Problèmes $\mathcal{NP}$ -complets

Un langage L est $\mathcal{NP}$ -complet si

- $L \in \mathcal{NP}$
- pour tout langage $L' \in \mathcal{NP}$, $L' \alpha L$.

S'il existe un langage $\mathcal{NP}$ -complet décidable par un algorithme polynomial, alors tous les langages de $\mathcal{NP}$ sont décidables en temps polynomial, c'est à dire $\mathcal{P} = \mathcal{NP}$.

Pour l'instant, la question $\mathcal{P} \stackrel{?}{=} \mathcal{NP}$ est ouverte.

Pour démontrer qu'un langage est $\mathcal{NP}$ -complet, on peut montrer :

- $L \in \mathcal{NP}$
 - il existe $L' \in \mathcal{NP}$ -complet tel que $L' \leq L$.
- Mais pour cela, il faut connaître un premier problème $\mathcal{NP}$ -complet.

10.6 SAT

Le premier problème $\mathcal{NP}$ -complet que l'on étudie est un problème de logique : SAT.

La section suivante couvre quelques rappels sur le calcul booléen et le calcul propositionnel afin de comprendre le problème SAT.

10.6.1 Calcul booléen

Le calcul booléen opère sur un domaine à deux valeurs $\{vrai, faux\}$. Il comporte quatre opérations : la négation ($\neg$), la conjonction ($\wedge$), disjonction ($\vee$) et l'implication ($\Rightarrow$). La définition de ces opérations est donnée par les tables de vérité suivantes :

p	$\neg p$
<i>vrai</i>	<i>faux</i>
<i>faux</i>	<i>vrai</i>

p	q	$p \wedge q$
<i>vrai</i>	<i>vrai</i>	<i>vrai</i>
<i>vrai</i>	<i>faux</i>	<i>faux</i>
<i>faux</i>	<i>vrai</i>	<i>faux</i>
<i>faux</i>	<i>faux</i>	<i>faux</i>

p	q	$p \vee q$
<i>vrai</i>	<i>vrai</i>	<i>vrai</i>
<i>vrai</i>	<i>faux</i>	<i>vrai</i>
<i>faux</i>	<i>vrai</i>	<i>vrai</i>
<i>faux</i>	<i>faux</i>	<i>faux</i>

p	q	$p \Rightarrow q$
<i>vrai</i>	<i>vrai</i>	<i>vrai</i>
<i>vrai</i>	<i>faux</i>	<i>faux</i>
<i>faux</i>	<i>vrai</i>	<i>vrai</i>
<i>faux</i>	<i>faux</i>	<i>vrai</i>

À partir des valeurs de vérité, des opérateurs booléens et de parenthèses, on construit des expressions booléennes.

Exemple 36

$(vrai \vee faux) \wedge faux, vrai \Rightarrow (faux \vee vrai)$ sont des expressions booléennes.

On peut évaluer les expressions booléennes.

Exemple 37

$(vrai \vee faux) \wedge faux$ est *faux*, $vrai \Rightarrow (faux \vee vrai)$ est *vrai*.

10.6.2 Calcul propositionnel

La seule différence entre le calcul propositionnel et le calcul booléen est que les expressions du calcul propositionnel sont bâties à partir de variables propositionnelles plutôt que des valeurs de vérité. Chaque variable propositionnelle représente une valeur de vérité : soit *vrai*, soit *faux*. Exemple d'expression du calcul propositionnel : $(p \vee q) \wedge r$.

Une expression du calcul propositionnel n'a de valeur que si l'on affecte des valeurs de vérité aux variables qui y figurent. La fonction qui affecte les valeurs de vérité aux variables propositionnelles est appelée fonction d'interprétation.

Exemple 38

Pour la fonction d'interprétation $\{p \leftarrow \text{vrai}, q \leftarrow \text{faux}, r \leftarrow \text{vrai}\}$, $(p \vee q) \wedge r$ a la valeur *vrai*.

Une expression est **satisfaisable** s'il existe au moins une fonction d'interprétation qui la rend vraie. $(p \vee q) \wedge r$ est satisfaisable.

Une expression du calcul propositionnel est en **forme normale conjonctive** si elle est de la forme :

$$E_1 \wedge E_2 \wedge \dots \wedge E_n$$

où chaque E_i est une clause, c'est à dire une disjonction de variables propositionnelles ou de négations de variables propositionnelles, à savoir une expression de la forme :

$$x_1 \vee x_2 \vee \dots \vee x_m$$

où chaque x_j est un littéral, c'est à dire une expression de la forme p ou $\neg p$.

Exemple 39

$(p \vee q) \wedge (\neg r) \wedge (\neg p \vee r)$ est en forme normale conjonctive.

10.6.3 Théorème de Cook

Problème SAT

Données : Une expression du calcul en forme normale conjonctive

Question : Est-ce que cette expression est satisfaisable ?

Le théorème de Cook : **Le problème SAT est $\mathcal{NP}$ -complet**

La preuve, qui n'est pas présenté, montre l'existence d'une transformation polynomiale qui à tout mot ω et tout langage $L \in \mathcal{NP}$ associe une instance de SAT qui est satisfaisable si et seulement si $\omega \in L$.

Chapitre 11

Rangs et médians

Contents

11.1 Avec un tri	75
11.2 Diviser pour régner	76
11.3 Faire des paquets	78
11.3.1 L'algorithme du médian	78
11.3.2 Complexité de l'algorithme du médian	79

Dans ce chapitre, nous allons étudier différentes solutions algorithmiques au problème du rang :

Problème du rang

Données : un tableau de n entiers tous distincts et un entier i compris entre 1 et n

Question : quel est l'élément du tableau qui est plus grand que $i-1$ autres éléments du tableau exactement ?

Exemple 40

Voici une instance de ce problème : pour le tableau $[4, 7, 9, 3, 10, 56, 2]$ et l'entier 3, le résultat est 4.

11.1 Avec un tri

Une première solution consiste à trier le tableau puis à le parcourir en s'arrêtant à la $i^{\text{ème}}$ case. En terme de comparaisons, il y en a $\Theta(n \log(n))$ pour un tri bien choisi (tri par tas, présenté dans le chapitre suivant) et i pour le parcours. Donc au total la complexité en moyenne comme dans le pire des cas de cette solution est $\Theta(n \log(n))$ comparaisons.

11.2 Diviser pour régner

Une deuxième solution reprend l'idée du tri-rapide : on partitionne récursivement le tableau d'entrée. Mais cette fois-ci, après la partition, on ne s'occupe que d'une seule région.

Voici l'algorithme :

```
algorithme de recherche du rang
donnée tab: tableau de n entiers
        i: entier
début
    rang(tab, i, 0, n-1)
fin

avec

fonction rang(tab: tableau de n entiers, ieme: entier, debut:
entier, fin: entier)
variables k,q: entiers
debut
    si (debut=fin)
        alors
            retourner tab[debut];
        sinon
            q <- partitionner(tab,debut,fin);
            // k est le nombre d'éléments inférieurs ou égaux à x
            k <- q-debut+1;
            si (k=ieme)
                alors // on est tombé sur l'élément recherché
                    retourner tab[q];
            fin si
            si (ieme<k)
                alors // l'élément recherché est dans la première partition
                    retourner rang(tab,ieme,debut,q-1);
            fin si
            si (ieme>k)
                alors // l'élément recherché est dans la deuxième partition
                    retourner rang(tab,ieme-k,q+1,fin);
            fin si
        fin si
    fin si
fin

et
```

```

fonction partitionner(tab: tableau de n entiers, i: entier, j: entier):
entier
variable aux, x, l, m: entier
        b: booléen
début
    x <- tab[i];
    l <- i;
    m <- j;
    b <- vrai;
    tant que (b) faire
        tant que (tab[m] > x) faire
            m <- m-1;
        fin tant que
        tant que (tab[l] < x) faire
            l <- l+1;
        fin tant que
        si (l < m) alors
            début
                aux <- tab[l];
                tab[l] <- tab[m];
                tab[m] <- aux;
            fin
        sinon b <- faux;
        fin si
    fin tant que
    retourner l;
fin

```

Comme on l'a vu dans le TP1 sur le *Tri rapide*, la fonction **partitionner** opère sur la partie du tableau **tab** comprise entre l'élément d'indice **i** et celui d'indice **j**. Elle considère le premier élément de cette partie appelé pivot (**x**). Elle déplace les différents éléments de cette partie du tableau afin que tous les éléments plus petits que le pivot aient un indice inférieur au sien et tous les éléments plus grand, un indice supérieur. Elle retourne l'indice du pivot.

La fonction **rang** est appelée sur la partie du tableau **tab** comprise entre l'élément d'indice **debut** et celui d'indice **fin**, elle recherche l'élément de rang **ieme**.

Si le tableau **tab** est réduit à un élément (**debut=fin**), alors la fonction retourne l'unique valeur du tableau. Sinon, elle appelle **partitionner** sur la portion du tableau qu'elle considère. **partitionner** va séparer les éléments du tableau en deux parties :

les éléments plus grands que le pivot et ceux plus petit. On nomme q l'indice du pivot et k désigne le nombre d'éléments inférieur ou égal au pivot.

Si $k=ieme$, le pivot est l'élément recherché. Si $ieme < k$, l'élément recherché est dans la première partition, on rappelle donc **rang** sur cette première partition. Si $ieme > k$, l'élément recherché est dans la deuxième partition, on rappelle donc **rang** sur cette deuxième partition, en pensant à rechercher l'élément de rang $ieme-k$ (il faut enlever les $k-1$ éléments de la première partition et le pivot).

L'algorithme de **recherche du rang** appelle la fonction **rang** sur tout le tableau.

Cet algorithme a une complexité en moyenne qui est linéaire en nombre de comparaisons mais une complexité dans le pire des cas qui est $\Theta(n^2)$.

11.3 Faire des paquets

11.3.1 L'algorithme du médian

On appelle **médian** l'élément de rang $\frac{n+1}{2}$ si n est impair et l'élément de rang $\frac{n}{2}$ si n est pair.

Voici l'algorithme récursif du médian résolvant le problème du rang :

```
si (n=1)
  alors on retourne l'unique élément du tableau
sinon
  1. on divise les n éléments du tableau en n/5 groupes de 5
    éléments chacun et un éventuel dernier groupe constitué
    des n mod 5 éléments restants.
  2. on trouve le médian de chaque groupe de la façon suivante :
    on trie par insertion les éléments de chaque groupe et on
    prend le médian de chaque groupe trié.
  3. on rappelle l'algorithme sur le tableau constitué par
    l'ensemble des médians trouvés lors de l'étape précédente
    afin d'en trouver le médian. On nomme x cet élément.
  4. on partitionne le tableau entier autour de x : à la fin
    de la partition les éléments plus petits que x sont placés
    avant lui dans le tableau, les éléments plus grands après.
    On nomme k le nombre d'éléments plus petits.
  5. si (i = k+1)
    alors l'élément cherché est x.
```



```

    fin si
  si (i < k+1)
    alors
      on rappelle l'algorithme sur les éléments plus petits
      afin de trouver l'élément de rang i
    fin si
  si (i > k+1)
    alors
      on rappelle l'algorithme sur les éléments plus grands
      afin de trouver l'élément de rang i-k-1
    fin si
  fin si
fin si

```

11.3.2 Complexité de l'algorithme du médian

Nous allons déterminer la complexité en moyenne de cet algorithme pour l'opération élémentaire de comparaison entre éléments du tableau.

Nous commençons par déterminer le nombre maximum d'éléments inférieurs et supérieurs au pivot de la partition x pour savoir combien il y a d'appels récursifs au maximum lors de l'étape 5.

Pour déterminer le nombre maximum d'éléments inférieurs au pivot de la partition x , nous allons déterminer une borne inférieure pour le nombre d'éléments supérieurs au pivot de la partition x .

Il y a $n/5$ groupes de 5 éléments dans ce tableau. On détermine le médian de chacun d'eux à l'étape 2, soit $n/5$ médians. On trouve ensuite le médian des médians x à l'étape 3.

Au moins la moitié des médians trouvés à l'étape 2 sont supérieurs ou égal au médian des médians x . On écarte de ces $(n/5)/2$ médians, le médian des médians. On considère donc $n/10 - 1$ médians.

Pour les groupes de chacun de ces $n/10 - 1$ médians, il y a au moins 3 éléments plus grands que x (le médian du groupe et les 2 éléments plus grands que lui).

Au total, on a donc au moins $3(n/10 - 1)$ éléments du tableau plus grands que le médian des médians.

On en déduit qu'il y a au plus $n - 3(n/10 - 1)$ éléments du tableau plus petits que le médian des médians soit $7n/10 + 3$.

Le même raisonnement permet de déduire qu'il y a au plus $7n/10 + 3$ éléments du tableau plus grands que le médian.

Donc, la fonction **rang** est appelée sur au plus $7n/10 + 3$ éléments à l'étape 5.

Calcul de la complexité

1. La première étape ne nécessite pas de comparaison entre éléments.
2. Le nombre de comparaisons du tri par insertion d'un tableau d'au plus 5 éléments peut être borné par 10. On fait au plus $n/5 + 1$ tris par insertion sur des tableaux d'au plus 5 éléments à l'étape 2. On effectue donc au plus $10(n/5 + 1)$ comparaisons entre éléments du tableau à l'étape 2.
3. L'étape 3 rappelle l'algorithme sur les médians. Il nécessite donc $C(n/5)$ comparaisons si la taille du tableau est un multiple de 5, $C(n/5 + 1)$ comparaisons sinon (où $C(n)$ est le nombre de comparaisons pour un tableau de taille n).
4. L'étape 4 est la partition, au pire, l et m prennent toutes les valeurs comprises entre le premier indice du tableau et le dernier, ce qui fait au plus $2n$ valeurs. Pour chacune d'elle, il y a une comparaison entre éléments du tableau. Cela fait un total d'au plus $2n$ comparaisons pour cette étape.
5. Lors de l'étape 5, on peut éventuellement rappeler l'algorithme. On vient de voir que c'est au plus sur $7n/10 + 3$ éléments. Donc cette étape nécessite au plus $C(7n/10 + 3)$.

Lorsque $n > 10$, $7n/10 + 3 < n$ donc :

- $C(n) \leq \text{constante}$ si la taille du tableau est plus petite ou égale à 10.
- $C(n) \leq 2n + 10 + C(n/5) + 2n + C(7n/10 + 3)$ si la taille du tableau est un multiple de 5 et supérieur à 10.
- $C(n) \leq 2n + 10 + C(n/5 + 1) + 2n + C(7n/10 + 3)$ si la taille du tableau est supérieur à 10 mais n'est pas un multiple de 5.

On cherche une constante λ et une valeur $N > 10$ telles que $C(n) \leq 10\lambda n$.
Supposons qu'elles existent alors si $n > N$,

$$C(n) \leq 2n + 10 + 2\lambda n + 10\lambda + 2n + 7\lambda n + 30\lambda$$

On veut que

$$2n + 10 + 2\lambda n + 10\lambda + 2n + 7\lambda n + 30\lambda \leq 10\lambda n$$

Soit :

$$4n + 10 + 40\lambda \leq \lambda n$$

Si $n \geq 80$ et $\lambda \geq 10$,

$$4n + 10 + 40\lambda \leq 5n + \lambda n/2 \leq \lambda n/2 + \lambda n/2 = \lambda n$$

On choisit $c \geq 10$ telle que $C(\mathbf{n}) \leq 10c\mathbf{n}$ pour tout $\mathbf{n} < 80$, on aura alors également $C(\mathbf{n}) \leq 10c\mathbf{n}$ pour tout $\mathbf{n} \geq 80$ en reprenant le calcul précédent.

On en conclut que la complexité de cet algorithme en moyenne comme dans le pire des cas est linéaire.

Chapitre 12

Arbres binaires et tri par tas

Contents

12.1 Arbres binaires	82
12.1.1 Définitions	82
12.1.2 Taille et hauteur d'un arbre binaire	83
12.2 Les tas	84
12.2.1 Arbres partiellement ordonnés et tas	84
12.2.2 Deux opérations sur les tas	85
12.2.3 Propriété des tas	86
12.3 Tri par tas	87
12.3.1 Représentation d'un tas par un tableau	87
12.3.2 Algorithme du tri par tas	87
12.3.3 Analyse de la complexité en nombre de comparaisons	90

Dans ce chapitre, nous allons voir le tri par tas qui est un tri par comparaisons efficace. Pour pouvoir le comprendre, il faut auparavant introduire la notion de tas et d'arbre binaire. Les arbres binaires et plus généralement les structures arborescentes sont très utilisées en informatique. Ce chapitre ne présente que le nécessaire pour comprendre le tri par tas.

12.1 Arbres binaires

Cette partie présente succinctement des arbres binaires d'entiers.

12.1.1 Définitions

Un **arbre binaire** est soit l'arbre vide, noté $\emptyset$, soit un triplet (a, g, d) où a est un entier et g et d sont des arbres.

Cette définition est récursive.

Exemple 41

$\emptyset$, $(2, \emptyset, \emptyset)$, $(3, (4, \emptyset, \emptyset), (2, \emptyset, (8, \emptyset, \emptyset)))$ et $(3, (4, (12, \emptyset, \emptyset), \emptyset), (2, \emptyset, (8, (5, \emptyset, \emptyset), \emptyset)))$ sont des arbres.

Si (a, g, d) est un arbre, on nomme a sa **racine**, g son **fil gauche** et d son **fil droit**. On nomme également **fil gauche** (resp. droit) de a la racine de g (resp. la racine de d). Le contexte permet de savoir si on parle d'un arbre ou d'un entier quand on écrit fils droit et fils gauche.

Les arbres sont représentés graphiquement avec la racine en haut et les fils en bas. On ne représente pas les arbres vides.

Exemple 42

Les trois exemples autres que $\emptyset$ sont dessinés dans la figure 12.1.

L'arbre le plus à droite a pour racine 3, son fil gauche est $(4, (12, \emptyset, \emptyset), \emptyset)$ et son fil droit $(2, \emptyset, (8, (5, \emptyset, \emptyset), \emptyset))$.

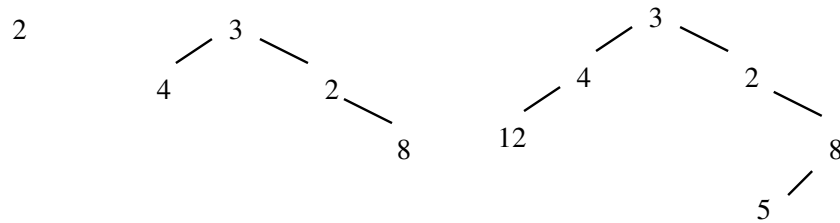


FIGURE 12.1 – Représentations graphiques d'arbres binaires.

Soient a et b , deux entiers, si b est le fils de a , on appelle a le **père** de b . De même, on définit la notion de **descendant** et d'**ascendant**. La racine et ses descendants sont appelés les **nœuds** de l'arbre. Une **feuille** est un nœud qui n'a pas de descendant.

Exemple 43

Dans la figure 12.1, 2 est l'unique feuille de l'arbre de gauche, 4 et 8 sont les feuilles de celui du milieu et 12 et 5 celles de celui de droite.

12.1.2 Taille et hauteur d'un arbre binaire

La **taille** d'un arbre est son nombre de nœuds, on la définit récursivement par :

- $taille(\emptyset) = 0$
- $taille((a, g, d)) = 1 + taille(g) + taille(d)$

Exemple 44

Les tailles des arbres de la figure 12.1 sont 1, 4 et 6.

La **hauteur** d'un arbre est défini récursivement par :

- $hauteur(\emptyset) = 0$
- $hauteur((a, g, d)) = 1 + \max(hauteur(g), hauteur(d))$

Exemple 45

Les hauteurs des arbres de la figure 12.1 sont 1, 3 et 4.

La hauteur et la taille d'un arbre binaire A sont liés par la propriété suivante :

$$\lfloor \log_2(taille(A)) \rfloor + 1 \leq hauteur(A) \leq taille(A)$$

12.2 Les tas

12.2.1 Arbres partiellement ordonnés et tas

Un **arbre partiellement ordonné** est un arbre dont les nœuds sont inférieurs ou égaux à leurs fils.

Un **arbre parfait** est un arbre binaire dont toutes les feuilles sont situées sur deux niveaux seulement, l'avant-dernier niveau étant complet et les feuilles du dernier niveau étant regroupées le plus à gauche possible.

Un **arbre binaire de recherche** est un arbre binaire dans lequel chaque nœud possède une clé, telle que chaque nœud du sous-arbre gauche ait une clé inférieure ou égale à celle du nœud considéré, et que chaque nœud du sous-arbre droit possède une clé supérieure ou égale à celle-ci.

Un **tas** est un arbre parfait partiellement ordonné.

Exemple 46

Les arbres de la figure 12.2 sont des tas.

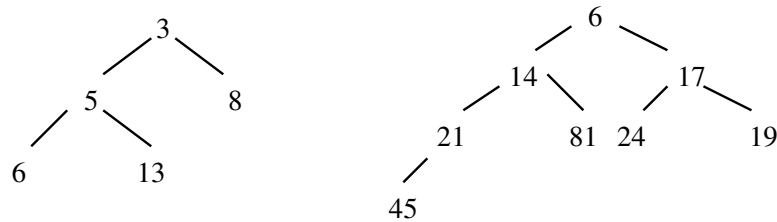


FIGURE 12.2 – Exemples de tas.

Exemple 47

Les arbres de la figure 12.3 ne sont pas des tas : dans celui de gauche le nœud 1 est le fils du nœud 5 donc il n'est pas partiellement ordonné. Dans l'arbre de droite,

24 a un fils alors que 81 qui est plus à gauche sur le même niveau n'en a pas. Cet arbre n'est donc pas parfait.

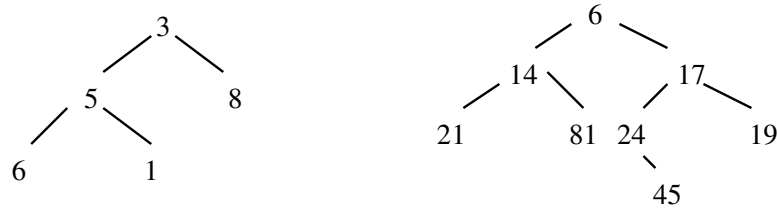


FIGURE 12.3 – Exemples d'arbres qui ne sont pas des tas.

12.2.2 Deux opérations sur les tas

12.2.2.1 Ajout d'un élément

L'ajout d'un nœud à un tas consiste à rajouter une feuille au dernier niveau de l'arbre (ou au niveau suivant si le dernier niveau est complet) le plus à gauche possible. On maintient ainsi l'arbre parfait. Mais l'arbre n'est plus forcément partiellement ordonné. Pour rétablir cette propriété, on échange la nouvelle feuille avec son père s'il est plus grand. Puis le père avec le nœud au dessus s'il est plus grand, ainsi de suite jusqu'à ce que la propriété soit rétablie.

Exemple 48

Cette manipulation est illustrée par la figure 12.4 où on ajoute 10 à un tas.

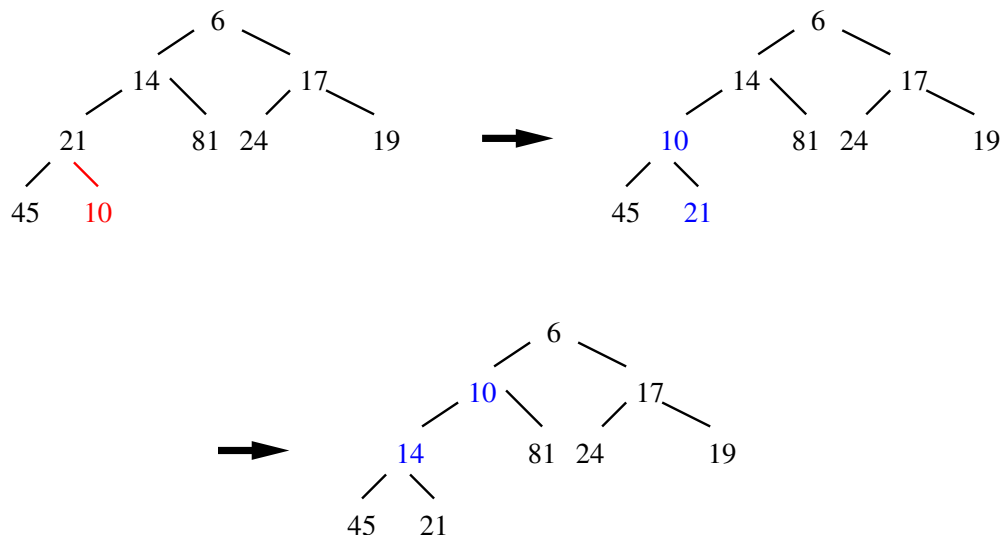


FIGURE 12.4 – Ajout d'une valeur à un tas.

On commence par placer 10 dans la feuille vacante tout en bas à gauche. Comme le père de 10 (21) est plus grand, on échange 10 et 21. On regarde au dessus, 10 et 14 ne sont pas dans le bon ordre, donc on les échange également. 10 et 6 sont ordonnés correctement donc on a fini.

12.2.2.2 Suppression du minimum

Pour supprimer le minimum d'un tas, il faut supprimer la racine de l'arbre. On commence par remplacer cette racine par la feuille la plus en bas à droite. On maintient ainsi l'arbre parfait mais il peut perdre la propriété d'ordre partiel. Pour la rétablir, on va échanger la racine avec le plus petit de ses fils, si l'un d'entre eux est plus petit qu'elle, puis recommencer avec le fils échangé. Ainsi de suite jusqu'à ce que l'arbre soit à nouveau partiellement ordonné.

Exemple 49

Cette manipulation est illustrée par la figure 12.5. On commence par mettre le 21 : la feuille la plus en bas à droite à la place de la racine. Ensuite, on met 10 à la place de 21 car 21 a un fils plus petit que lui et 10 est son fils le plus petit. Puis 19 à la place de 21 car 21 a un fils plus petit que lui et 19 est son fils le plus petit.

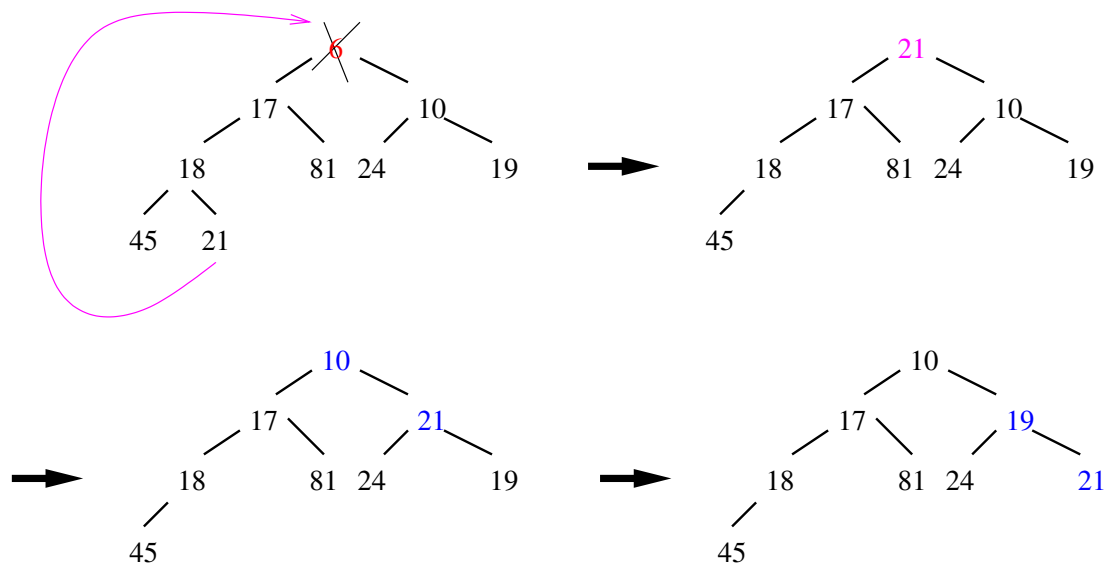


FIGURE 12.5 – Suppression du minimum d'un tas.

12.2.3 Propriété des tas

Si A est un arbre parfait ou un tas, il vérifie :

$$\text{hauteur}(A) = \lfloor \log_2(\text{taille}(A)) \rfloor + 1$$

12.3 Tri par tas

12.3.1 Représentation d'un tas par un tableau

On peut très facilement représenter un tas par un tableau, il suffit de lire les nœuds de l'arbre de haut en bas et de gauche à droite.

Exemple 50

Les arbres de la figure 12.2 sont représentés par les tableaux $[3, 5, 8, 6, 13]$ et $[6, 14, 17, 21, 81, 24, 19, 45]$.

Si t est un tableau qui représente un tas et p est sa taille :

- $t[0]$ est la racine
- $t[i/2 - 1]$ est le père de $t[i - 1]$.
- $t[i]$ a deux fils s'ils existent $t[2 * i + 1]$ et $t[2 * i + 2]$.
- si $i \geq p/2$, $t[i]$ est une feuille

12.3.2 Algorithme du tri par tas

Le tri par tas proposé permet de trier par ordre décroissant un tableau. Le principe est le suivant, il ressemble un peu au tri par sélection : on construit un tas contenant les n éléments du tableau à trier par des ajouts successifs. On supprime successivement le minimum que l'on met à la fin du tableau après l'avoir réordonné.

```
algorithme tri_tas
donnee t: tableau de n entiers
variables p,min: entiers
debut
  p <- 0;
  tant que (p<n)
    ajouter(t[p],t);
  fin tant que
  tant que (p>1)
    min <- supprimerMin(t);
    t[p] <- min;
  fin tant que
fin

procedure ajouter(x: entier, tab: tableau de n entiers)
variables i,aux: entiers
        b: booleen
debut
```

```

p <- p+1;
tab[p-1] <- x;
i <- p;
b <- (i>1);
si (b)
  alors
    b <- (tab[i-1] < tab[(i/2)-1]);
fin si
tant que (b)
  aux <- tab[i-1];
  tab[i-1] <- tab[(i/2)-1];
  tab[(i/2)-1] <- aux;
  i <- i/2;
  b <- (i>1);
  si (b)
    alors
      b <- (tab[i-1] < tab[(i/2)-1]);
  fin si
fin tant que
fin

fonction supprimerMin(tab: tableau de n entiers):entier
variables i,j,min: entiers
debut
  min <- tab[0];
  tab[0] <- tab[p-1];
  p <- p-1;
  i <- 1;
  tant que (i<=(p/2))
    si ((2*i==p) ou (tab[2*i-1] < tab[2*i]))
      alors
        j <- 2*i;
      sinon
        j <- 2*i+1;
    fin si
    si (tab[i-1] > tab[j-1])
      alors
        echanger(tab[i-1],tab[j-1]);
        i <- j;
      sinon

```

```

    retourner min;
  fin si
fin tant que
retourner min;
fin

```

Exemple 51

Pour le tableau $t=[10, 11, 3, 6, 4, 2]$, la première partie de cet algorithme (construction du tas) est proposée dans la figure 12.6 et la seconde (suppression des minimums successifs) dans la figure 12.7.

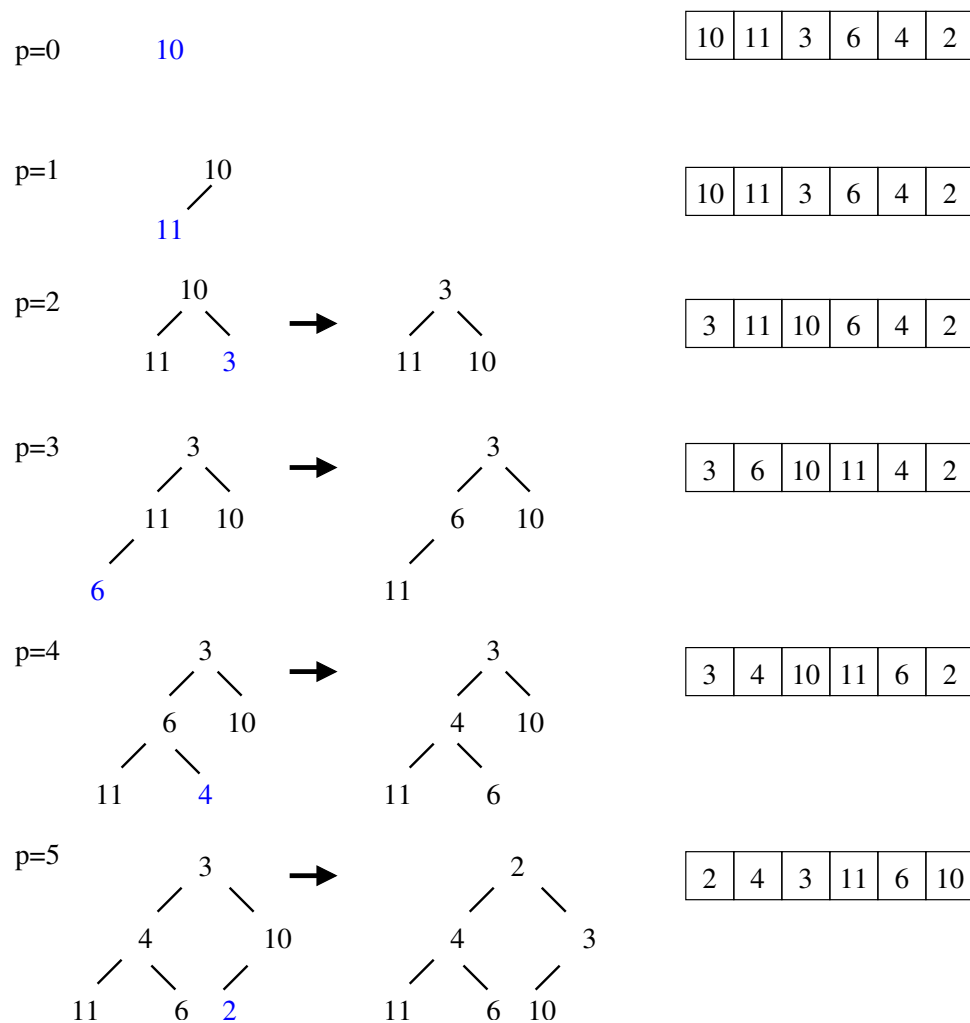


FIGURE 12.6 – Tri par tas de $t=[10, 11, 3, 6, 4, 2]$ (construction du tas).

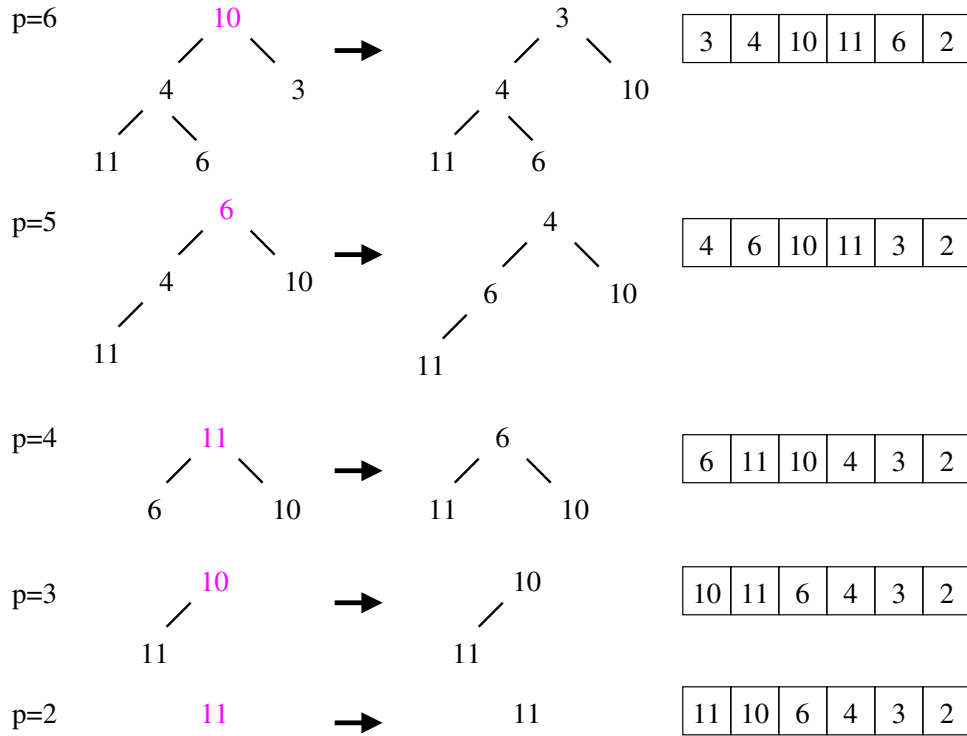


FIGURE 12.7 – Tri par tas de $t = [10, 11, 3, 6, 4, 2]$ (suppression des minimums successifs).

12.3.3 Analyse de la complexité en nombre de comparaisons

Le nombre de comparaisons entre deux éléments du tableau lors de l'exécution du tri par tas dépend des valeurs des éléments du tableau. Les comparaisons ont lieu lors de la procédure **ajouter** et de la fonction **supprimerMin**.

- Lors de l'exécution de la procédure **ajouter**, on parcourt au plus une branche complète de l'arbre, en faisant une comparaison par nœud. On fait donc au plus “hauteur de l'arbre” comparaisons.
- Lors de l'exécution de la fonction **supprimerMin**, on parcourt au plus une branche complète de l'arbre, en faisant deux comparaisons par nœud. On fait donc au plus “deux hauteurs de l'arbre” comparaisons.

Le tri par tas appelle la procédure **ajouter** pour chacune des valeurs de p entre 0 et $n - 1$, soit n appels. La hauteur de l'arbre est majorée par la hauteur du tas complet soit $\lfloor \log_2 n \rfloor + 1$ (rappelons le résultat du paragraphe 12.2.3 : si A est un arbre parfait ou un tas, alors $\text{hauteur}(A) = \lfloor \log_2(\text{taille}(A)) \rfloor + 1$). En conclusion, les appels à **ajouter** demandent moins de $n \log_2 n + n$ comparaisons.

Le tri par tas appelle la fonction **supprimerMin** pour chacune des valeurs de

p entre $n - 1$ et 1 , soit $n - 1$ appels. Par conséquent, les appels à **supprimerMin** demandent moins de $2n \log_2 n + 2n$ comparaisons.

Il s'ensuit que la complexité dans le pire des cas en nombre de comparaisons du tri par tas est $O(n \log_2 n)$. D'où le résultat :

$$C_{moy}(n) = C_{pire}(n) = O(n \log_2 n)$$